



TRABAJO FIN DE GRADO
GRADO EN INGENIERÍA INFORMÁTICA
MENCIÓN EN INGENIERÍA DEL SOFTWARE



Diseño, despliegue y operación de una plataforma cloud para la gestión de transportes en AWS mediante infraestructura como código y prácticas DevOps

Estudiante: Pablo Manzanares López

Dirección: Paula María Castro Castro

A Coruña, mayo de 2026.

Agradecimientos

A mi familia, amistades y profesorado por el apoyo recibido durante el desarrollo de este trabajo. Gracias por el apoyo, la paciencia y el acompañamiento durante todo el proceso.

Resumen

Este Trabajo Fin de Grado aborda el diseño, desarrollo, despliegue y operación de una plataforma cloud para la gestión de transportes sobre AWS. El proyecto se plantea como un backend de alcance funcional acotado, pero con una carga técnica significativa en arquitectura, automatización, infraestructura como código, seguridad básica y validación operativa.

La solución implementa una API REST en ASP.NET Core sobre PostgreSQL para gestionar transportes, vehículos, conductores, usuarios, asignaciones, transiciones de estado y eventos logísticos. La aplicación se empaqueta mediante Docker y se despliega en AWS usando Terraform, ECS Fargate, RDS PostgreSQL, ECR, Application Load Balancer, Route 53, ACM, Secrets Manager, SSM Parameter Store y CloudWatch. También se ha definido un flujo de despliegue manual desde GitHub Actions con autenticación OIDC, evitando claves AWS persistentes.

Como resultado, se ha obtenido un entorno dev real recreable y validado en AWS, con servicio ECS estable, migraciones EF Core ejecutadas como tarea puntual de ECS, verificación de salud, bootstrap de administrador, login con JWT, logs JSON correlables en CloudWatch, dashboard operativo, alarmas Terraform para ALB/ECS/RDS/API y configuración DNS/HTTPS mediante Route 53 y ACM. Tras capturar la evidencia operativa, el entorno dev se destruyó con Terraform para evitar costes residuales, conservando únicamente recursos base de bajo coste como el dominio y la zona DNS.

Abstract

This Bachelor's Thesis addresses the design, development, deployment, and operation of a cloud-based transport management platform on AWS. The project is implemented as a deliberately scoped backend, with a strong technical focus on architecture, automation, infrastructure as code, basic security, and operational validation.

The solution provides a REST API built with ASP.NET Core and PostgreSQL to manage transports, vehicles, drivers, users, assignments, state transitions, and logistics events. The application is packaged with Docker and deployed to AWS using Terraform, ECS Fargate, RDS PostgreSQL, ECR, Application Load Balancer, Route 53, ACM, Secrets Manager, SSM Parameter Store, and CloudWatch. A manual GitHub Actions deployment flow with OIDC authentication has also been defined, avoiding persistent AWS access keys.

The resulting dev environment is reproducible and has been validated on AWS with a stable ECS service, one-off EF Core migrations, health checks, first-admin bootstrap, JWT login, JSON structured logs with request correlation, a CloudWatch dashboard, Terraform-managed alarms for ALB/ECS/RDS/API, and DNS/HTTPS configuration through Route 53

and ACM. After collecting operational evidence, the dev stack was destroyed with Terraform to avoid residual cloud costs, leaving only low-cost foundational resources such as the domain and DNS zone.

Palabras clave:

- Computación en la nube
- DevOps
- Infraestructura como código
- AWS
- Terraform
- ASP.NET Core
- Gestión de transportes

Keywords:

- Cloud computing
- DevOps
- Infrastructure as code
- AWS
- Terraform
- ASP.NET Core
- Transport management

Índice general

1	Introducción	1
1.1	Motivación	1
1.2	Problema a resolver	2
1.3	Alcance del trabajo	2
1.4	Estructura de la memoria	2
2	Contexto y objetivos	3
2.1	Contexto tecnológico	3
2.2	Objetivo general	3
2.3	Objetivos específicos	4
2.4	Criterios de éxito	4
3	Marco tecnológico	6
3.1	Backend API y arquitectura de servicios	6
3.2	Persistencia relacional y migraciones	7
3.3	Contenedores y reproducibilidad	7
3.4	Infraestructura como código	8
3.5	AWS como plataforma de despliegue	9
3.6	CI/CD y entrega controlada	9
3.7	Observabilidad mínima	10
3.8	Seguridad y coste como restricciones de diseño	10
4	Metodología y planificación	12
4.1	Enfoque de trabajo	12
4.2	Fases del proyecto	12
4.3	Gestión del alcance	12
4.4	Ejecución por sprints	13
4.5	Herramientas utilizadas	14

5	Análisis y requisitos	15
5.1	Actores	15
5.2	Requisitos funcionales	15
5.3	Requisitos no funcionales	16
5.4	Modelo de dominio	17
5.5	Casos de uso principales	17
6	Diseño de la arquitectura	19
6.1	Visión general	19
6.2	Arquitectura lógica del backend	19
6.3	Arquitectura de datos	20
6.4	Arquitectura cloud en AWS	20
6.5	Decisiones arquitectónicas	21
7	Diseño detallado del sistema	23
7.1	Criterios de diseño	23
7.2	Diseño del contrato HTTP	23
7.3	Diseño del dominio	24
7.4	Diseño de persistencia	25
7.5	Diseño de autenticación y autorización	25
7.6	Diseño de configuración	26
7.7	Diseño de despliegue	26
7.8	Diseño de validación	27
7.9	Decisiones descartadas	27
8	Implementación del backend	29
8.1	Estructura del proyecto	29
8.2	API REST	29
8.3	Persistencia	30
8.4	Autenticación y autorización	30
8.5	Pruebas del backend	30
9	Infraestructura y despliegue	32
9.1	Contenerización	32
9.2	Infraestructura como código	32
9.3	Entorno AWS	33
9.4	Despliegue de la aplicación	34
9.5	Rollback y restore	35
9.6	DNS, certificados y HTTPS	35

9.7	Cierre del entorno y control de costes	36
9.8	Gestión de configuración y secretos	37
10	Integración y despliegue continuo	38
10.1	Objetivos del pipeline	38
10.2	Validación automática	38
10.3	Publicación de artefactos	39
10.4	Despliegue automatizado	39
11	Observabilidad y seguridad	42
11.1	Comprobaciones de salud	42
11.2	Logs y métricas	42
11.3	Alarmas y operación	43
11.4	Seguridad aplicada	44
11.5	Limitaciones de seguridad	45
11.6	Revisión de seguridad Sprint 8	45
12	Operación y runbooks	46
12.1	Objetivo operativo	46
12.2	Runbook de despliegue normal	46
12.3	Runbook de rollback	47
12.4	Runbook de reinicio o redeploy	47
12.5	Runbook de bootstrap de administrador	48
12.6	Runbook de restore RDS	48
12.7	Runbook de logs por correlation id	49
12.8	Runbook de alarmas	49
12.9	Runbook de destroy y auditoría	50
12.10	Aprendizajes operativos	50
13	Validación y resultados	51
13.1	Estrategia de validación	51
13.2	Pruebas funcionales	51
13.3	Validación de infraestructura	52
13.4	Resultados obtenidos	52
13.5	Validación Sprint 7	53
13.6	Evidencia de cierre de coste	54
13.7	Validación Sprint 8	56
13.8	Análisis de limitaciones	56
13.9	Cierre Sprint 9	58

14 Conclusiones	59
14.1 Conclusiones del trabajo	59
14.2 Competencias aplicadas	59
14.3 Lecciones aprendidas	60
14.4 Líneas futuras	60
A Material complementario	63
A.1 Anteproyecto	63
A.2 Evidencias técnicas	63
B Trazabilidad de requisitos y evidencias	65
B.1 Propósito del anexo	65
B.2 Requisitos funcionales	65
B.3 Requisitos no funcionales	66
B.4 Evidencias por sprint	67
B.5 Relación entre evidencia y defensa	68
B.6 Capturas incorporadas	68
C Procedimientos operativos	70
C.1 Recreación completa de dev	70
C.2 Comprobaciones locales previas	70
C.3 Carga de secretos runtime	71
C.4 Migraciones ECS	71
C.5 Validación HTTPS y DNS	72
C.6 Auditoría de seguridad	72
C.7 Auditoría de coste	72
C.8 Guion de defensa técnica	73
C.9 Checklist de cierre	73
C.10 Diagnóstico por capas	74
C.11 Criterios de aceptación final	75
C.12 Uso de los TFG de referencia	75
C.13 Mantenimiento posterior	76
D Comparativa estructural con memorias de referencia	77
D.1 Criterios observados	77
D.2 Ajustes aplicados a TransitOps	77
D.3 Resultado de la comparación	78
Lista de acrónimos	79

Glosario	80
Bibliografía	81

Índice de figuras

6.1	Evidencia DNS para el endpoint de desarrollo.	21
9.1	Evidencia de rollback automático y restore real de RDS.	35
9.2	Evidencia del certificado ACM emitido para el endpoint de desarrollo.	36
10.1	Evidencia de despliegue cloud final: imagen publicada en ECR, servicio ECS estable y readiness HTTPS con respuesta 200.	40
10.2	Verificación de la task definition activa apuntando al tag operativo validado. .	40
10.3	Outputs de Terraform tras la recreación del entorno dev, incluyendo ALB, DNS, ECS y task definition activa.	41
11.1	Evidencia de petición readiness con identificador de correlación y log JSON correlable en CloudWatch.	43
11.2	Evidencia del dashboard operativo de TransitOps con métricas de ALB, ECS, RDS y logs recientes.	44
11.3	Evidencia de revisión de seguridad Sprint 8.	45
13.1	Evidencia de servicio cloud operativo: ECS estable y readiness HTTPS con respuesta 200.	55
13.2	Evidencia de revisión de coste Sprint 8.	57
13.3	Evidencia de auditoría post-destroy Sprint 8.	57

Índice de cuadros

5.1	Requisitos funcionales principales	15
A.1	Capturas y evidencias incorporadas	63
B.1	Trazabilidad de requisitos funcionales	65
B.2	Trazabilidad de requisitos no funcionales	66
B.3	Resumen de evidencias por sprint	67

Introducción

Este trabajo presenta el diseño, desarrollo, despliegue y operación de una plataforma cloud para la gestión de transportes. El proyecto se plantea como un trabajo clásico de ingeniería en el que el alcance funcional se mantiene deliberadamente acotado para poder profundizar en decisiones de arquitectura, automatización, [infraestructura como código](#), observabilidad y operación en [Amazon Web Services \(AWS\)](#).

1.1 Motivación

El desarrollo de software actual no se limita a escribir lógica de negocio. Un sistema backend debe poder compilarse, probarse, configurarse, desplegarse, observarse y recuperarse de forma repetible. Esta realidad hace que la ingeniería del software moderna incorpore prácticas de DevOps, infraestructura como código, contenedores, servicios gestionados y automatización de despliegues.

El dominio de gestión de transportes resulta adecuado para este trabajo porque permite modelar entidades y reglas de negocio reales sin convertir el proyecto en un sistema funcionalmente excesivo. Transportes, vehículos, conductores, usuarios, estados y eventos logísticos ofrecen suficiente complejidad para demostrar diseño de API, persistencia, validación, seguridad y trazabilidad, pero mantienen el alcance dentro de un tamaño razonable para un Trabajo Fin de Grado.

La motivación principal del proyecto es construir un sistema pequeño en funcionalidad, pero serio en operación. En lugar de ampliar indefinidamente las características de producto, el trabajo prioriza demostrar que una API backend puede pasar de código fuente a un entorno cloud real en AWS de forma reproducible y documentada.

1.2 Problema a resolver

El problema abordado consiste en proporcionar un backend que permita a un equipo operativo gestionar transportes y sus recursos asociados. El sistema debe permitir crear y consultar transportes, mantener vehículos y conductores, asignar recursos, controlar el ciclo de vida de cada transporte y registrar eventos que aporten trazabilidad.

Además del problema funcional, el trabajo aborda un problema de ingeniería: cómo empaquetar, desplegar y operar esa API en un entorno cloud sin depender de pasos manuales frágiles ni de infraestructura creada de forma informal. Para ello, la infraestructura se define con Terraform, la aplicación se ejecuta como contenedor, la base de datos se despliega como servicio gestionado y los secretos se externalizan.

1.3 Alcance del trabajo

El trabajo se centra en un [backend API](#), persistencia relacional, ejecución local reproducible, infraestructura AWS definida con Terraform, pipeline [CI/CD](#) y mecanismos básicos de observabilidad y seguridad. Quedan fuera del alcance principal el desarrollo de una interfaz de usuario completa y las funcionalidades avanzadas de planificación logística.

1.4 Estructura de la memoria

La memoria se organiza en catorce capítulos y varios anexos. Tras esta introducción, el capítulo [2](#) presenta el contexto y los objetivos del trabajo. El capítulo [3](#) resume las tecnologías utilizadas y justifica su papel dentro del proyecto. El capítulo [4](#) describe la metodología y la planificación seguida. El capítulo [5](#) recoge requisitos, actores y reglas del dominio. Los capítulos [6](#) y [7](#) explican la arquitectura general y el diseño técnico detallado. El capítulo [8](#) detalla la implementación del backend. Los capítulos [9](#), [10](#), [11](#) y [12](#) tratan infraestructura, despliegue, automatización, observabilidad, seguridad y operación. El capítulo [13](#) recoge la validación realizada y los resultados obtenidos. Finalmente, el capítulo [14](#) presenta conclusiones, competencias aplicadas y líneas futuras. Los anexos recogen evidencias, trazabilidad y procedimientos operativos.

Contexto y objetivos

2.1 Contexto tecnológico

Las aplicaciones backend actuales suelen desplegarse en entornos distribuidos, contenerizados y gestionados por proveedores cloud. Esto permite reducir la administración directa de servidores, pero introduce nuevas responsabilidades: definir redes, permisos, secretos, balanceadores, bases de datos, registros de imágenes, observabilidad y procesos de despliegue.

En este contexto, la **infraestructura como código** permite describir la infraestructura mediante ficheros versionables, revisables y repetibles. Terraform se utiliza en este proyecto para codificar la red, la base de datos, los servicios de ejecución, los repositorios de imágenes, el balanceador de carga, la configuración de DNS y los permisos necesarios [1].

La contenerización proporciona una unidad de despliegue estable entre el entorno local y AWS. La API se empaqueta en una imagen Docker y se ejecuta en ECS Fargate, evitando la gestión directa de servidores. La persistencia se apoya en PostgreSQL gestionado mediante RDS, y el acceso público se concentra en un Application Load Balancer protegido por HTTPS.

El enfoque DevOps aparece en la conexión entre código, validación automática, construcción de imagen, publicación en ECR, despliegue Terraform, migraciones de base de datos y pruebas de humo. El proyecto busca que estas piezas formen un camino verificable desde el repositorio hasta un entorno real.

2.2 Objetivo general

Diseñar, implementar, desplegar y operar una plataforma cloud para la gestión de transportes sobre AWS, utilizando infraestructura como código y prácticas DevOps para garantizar reproducibilidad, automatización, seguridad básica y capacidad de observación.

2.3 Objetivos específicos

- Diseñar una arquitectura cloud para una plataforma de gestión de transportes sobre AWS.
- Implementar un backend con funcionalidades básicas de transportes, vehículos, conductores, usuarios y eventos.
- Definir la infraestructura mediante Terraform para obtener entornos reproducibles.
- Contenerizar la aplicación y desplegarla en servicios gestionados de AWS.
- Automatizar validación, construcción y despliegue mediante CI/CD.
- Incorporar logs, métricas, comprobaciones de salud y trazabilidad operativa.
- Aplicar medidas básicas de seguridad en autenticación, autorización, secretos e infraestructura.
- Evaluar el sistema mediante pruebas funcionales y análisis de ejecución.
- Documentar decisiones, limitaciones y líneas de evolución futura.

2.4 Criterios de éxito

Los criterios de éxito se han formulado como resultados observables:

- La solución compila y las pruebas automatizadas principales se ejecutan correctamente.
- La API permite gestionar el flujo operativo básico: autenticación, usuarios, transportes, vehículos, conductores, asignación, cambios de estado y eventos.
- La base de datos se gestiona mediante migraciones de Entity Framework Core y no mediante cambios manuales no versionados.
- El entorno local puede arrancarse con Docker Compose y configuración externa.
- La infraestructura de AWS se define en Terraform y usa estado remoto S3 con bloqueo DynamoDB.
- La aplicación se despliega como contenedor en ECS Fargate, detrás de ALB y con persistencia en RDS PostgreSQL.
- El entorno dev expone un endpoint HTTPS real en `api.dev.transitops.net`.

- Las migraciones en AWS se ejecutan como tarea ECS puntual, no como efecto lateral opaco del arranque web.
- Los secretos no se almacenan en el repositorio y se consumen desde Secrets Manager.
- Las peticiones son correlables mediante X-Correlation-ID y los logs JSON pueden consultarse en CloudWatch.
- La infraestructura crea dashboard y alarmas operativas mínimas para ALB, ECS, RDS y errores de aplicación.
- El entorno dev puede destruirse con Terraform para eliminar recursos costosos tras capturar evidencias.
- Existe documentación suficiente para explicar arquitectura, despliegue, validación y limitaciones.

Marco tecnológico

3.1 Backend API y arquitectura de servicios

El proyecto se apoya en una arquitectura backend centrada en una API REST. Esta elección encaja con el objetivo del trabajo por varias razones. En primer lugar, una API HTTP permite separar claramente el contrato público del sistema de su implementación interna. En segundo lugar, facilita la validación mediante herramientas estándar como Postman, Newman, curl o pruebas de integración. Por último, permite desplegar el servicio detrás de un balanceador de carga y tratar la aplicación como una unidad *stateless*, lo que simplifica el despliegue sobre contenedores.

ASP.NET Core proporciona el modelo de ejecución HTTP, el sistema de inyección de dependencias, la configuración por entornos, el soporte para middlewares y la integración con autenticación JWT. En TransitOps se usa de forma deliberadamente contenida: la solución no introduce una arquitectura distribuida ni varios microservicios porque el dominio funcional no lo requiere. El interés principal está en demostrar que un backend monolítico, bien estructurado y desplegado correctamente, puede alcanzar una calidad operativa suficiente para un entorno cloud real.

El diseño de la API sigue una separación por responsabilidades: controladores para recibir peticiones, contratos para definir entradas y salidas, servicios de aplicación para coordinar casos de uso, entidades de dominio para representar reglas centrales e infraestructura para persistencia. Esta estructura permite razonar sobre el sistema sin imponer capas artificiales. En un proyecto académico de alcance acotado, una división excesiva en proyectos o servicios habría añadido coste de mantenimiento sin aportar evidencia técnica relevante.

El uso de HTTP también facilita definir criterios de aceptación observables. Un endpoint de salud debe responder con un código concreto; un login válido debe devolver un token; una operación protegida debe rechazar al usuario no autorizado; un error de validación debe devolver un estado coherente. Estos comportamientos se pueden automatizar en pruebas y

repetir tanto en local como en AWS.

3.2 Persistencia relacional y migraciones

TransitOps utiliza PostgreSQL como base de datos relacional [2]. El dominio del proyecto incluye usuarios, transportes, vehículos, conductores, asignaciones y eventos. Estas entidades presentan relaciones claras y restricciones de consistencia que encajan bien con un modelo relacional. Por ejemplo, un evento logístico pertenece a un transporte, un transporte puede tener asignados un vehículo y un conductor, y determinadas referencias deben ser únicas entre registros activos.

El uso de Entity Framework Core permite modelar estas entidades desde el código .NET y mantener el esquema mediante migraciones versionadas. La migración no se trata como un paso manual improvisado, sino como parte del ciclo de vida del sistema. En local puede ejecutarse de forma automática al arrancar el entorno de desarrollo, mientras que en AWS se ejecuta como tarea ECS puntual mediante el modo `--migrate-only`. Esta separación evita mezclar el arranque del servicio web con cambios de esquema en producción o en entornos compartidos.

La decisión de usar migraciones también aporta trazabilidad. Cada cambio de esquema queda registrado en el repositorio y puede revisarse junto al código que lo necesita. Esto reduce el riesgo de que el estado de la base de datos dependa de instrucciones manuales no documentadas. Además, permite validar que una base restaurada desde snapshot puede aceptar las migraciones pendientes, como se comprobó en Sprint 7 con la prueba de restore sobre una instancia temporal.

PostgreSQL aporta restricciones a nivel de base de datos que complementan la validación de aplicación. En TransitOps se usan índices únicos condicionados por borrado lógico, tipos enteros para estados controlados y relaciones entre tablas. La aplicación sigue siendo responsable de expresar reglas de negocio y devolver errores comprensibles, pero la base de datos actúa como última línea de defensa para la integridad persistente.

3.3 Contenedores y reproducibilidad

Docker se utiliza para empaquetar la API y para reproducir el entorno local [3]. Esta decisión responde a una necesidad práctica: reducir diferencias entre la máquina de desarrollo, el pipeline de integración y el entorno cloud. La imagen Docker define el runtime necesario para ejecutar la aplicación, instala únicamente dependencias imprescindibles y configura el proceso para escuchar en el puerto esperado por ECS.

En local, Docker Compose levanta la API y PostgreSQL con una configuración controlada.

Esto permite ejecutar pruebas de humo contra una API viva sin instalar manualmente la base de datos ni depender de servicios externos. Para el proyecto, esta capacidad es importante porque conecta el desarrollo backend con la ejecución cloud: se valida el comportamiento dentro de contenedor antes de publicar una imagen en ECR.

La imagen de la API se endureció durante Sprint 7. El contenedor se ejecuta con usuario no root, expone únicamente el puerto necesario y usa una imagen base de runtime en lugar de una imagen completa de SDK. También se añadió un fichero `.dockerignore` para evitar enviar al daemon de Docker carpetas de compilación, metadatos de Git, directorios de Terraform, ficheros locales y artefactos pesados de documentación. Esta mejora no cambia la funcionalidad, pero reduce superficie accidental y acelera el proceso de build.

El etiquetado de imágenes es parte de la trazabilidad del despliegue. En los sprints cloud se usaron tags operativos como `sprint6-local-6bb910c`, `sprint7-good-3dca035` o `sprint8-good-fdf98b9`. Estos tags permiten explicar qué versión de la API se ejecutó en cada validación y facilitan el rollback a una imagen conocida. La imagen no es solo un artefacto técnico, sino una pieza de evidencia del flujo de entrega.

3.4 Infraestructura como código

Terraform se utiliza para definir la infraestructura AWS. Frente a una configuración manual desde consola, la infraestructura como código ofrece varias ventajas: versionado, revisión de cambios, reproducibilidad, posibilidad de destrucción controlada y documentación ejecutable de la arquitectura. En TransitOps, esta decisión es central porque el objetivo no es únicamente tener una API funcionando, sino poder recrear el entorno dev desde cero con pasos defendibles.

El proyecto separa módulos reutilizables y raíz de entorno. Los módulos encapsulan piezas como red, registro de contenedores, base de datos, runtime ECS, configuración, observabilidad y OIDC de GitHub. La raíz `environments/dev` decide qué módulos se usan y con qué variables. Esta estructura permite mantener el entorno de desarrollo concreto sin duplicar lógica de infraestructura.

El estado remoto se almacena en S3 y se bloquea mediante DynamoDB. Este punto es importante porque Terraform necesita conocer qué recursos gestiona. Si el estado se quedase únicamente en local, sería fácil perderlo, divergir entre máquinas o aplicar cambios concurrentes. El backend remoto hace que la infraestructura sea más reproducible y reduce el riesgo de errores humanos durante los sprints de validación.

El uso de Terraform también hace explícitos los tradeoffs del entorno dev. Por ejemplo, se mantiene `desired_count = 0` cuando se desea crear infraestructura sin arrancar todavía la API, se desactivan backups automáticos de RDS para evitar coste recurrente, se permite borrar ECR

con imágenes en desarrollo y se destruye el stack al terminar cada validación. Estas decisiones no serían tan visibles si los recursos se creasen manualmente.

3.5 AWS como plataforma de despliegue

AWS se utiliza como plataforma cloud porque ofrece servicios gestionados que cubren las necesidades del proyecto sin obligar a administrar servidores. ECS Fargate ejecuta contenedores sin gestionar instancias EC2; ECR almacena imágenes Docker; RDS PostgreSQL ofrece base de datos gestionada; ALB expone la API; Route 53 y ACM resuelven el dominio y el certificado TLS; CloudWatch centraliza logs, métricas y alarmas; Secrets Manager y SSM Parameter Store almacenan configuración sensible y parámetros [4, 5, 6, 7, 8, 9, 10, 11].

La arquitectura desplegada sigue un patrón habitual para APIs web: balanceador público en subredes públicas, servicio ECS en subredes privadas y base de datos en subredes privadas de datos. El tráfico permitido queda limitado a Internet hacia ALB, ALB hacia ECS y ECS hacia RDS. De este modo, la API se expone por HTTPS, pero la tarea de aplicación y la base de datos no quedan directamente accesibles desde Internet.

ECS Fargate encaja con el alcance del trabajo porque evita gestionar clústeres de máquinas. La unidad operativa pasa a ser la task definition: qué imagen ejecutar, qué variables y secretos inyectar, qué recursos reservar, qué puerto exponer y cómo enviar logs. El servicio ECS mantiene el número deseado de tareas y se integra con el target group del ALB para recibir tráfico solo cuando la ruta de readiness responde correctamente.

RDS aporta una base de datos gestionada que simplifica operación. Aun así, el proyecto no ignora la recuperación: Sprint 7 añadió una prueba real de snapshot manual, restauración temporal y migración contra la base restaurada. Esto demuestra que la decisión de desactivar backups recurrentes en dev es una optimización de coste para un entorno efímero, no una ausencia de criterio sobre protección de datos.

3.6 CI/CD y entrega controlada

GitHub Actions se usa para validar y desplegar de forma manual [12]. La ejecución manual es intencionada: durante el desarrollo del TFG se prioriza capturar evidencias controladas y evitar despliegues accidentales. El pipeline automatiza pasos repetibles como restaurar dependencias, compilar, ejecutar tests, construir imagen Docker, publicar en ECR, aplicar Terraform, ejecutar migraciones, escalar ECS y validar health/login.

El acceso a AWS desde GitHub se diseña mediante OIDC. Esto evita almacenar claves AWS permanentes como secretos de larga duración. El workflow obtiene un token federado y asume un rol IAM limitado por la trust policy al repositorio y rama configurados. En Sprint

8 se documentó que el rol mantiene permisos amplios sobre los servicios gestionados por Terraform para poder aplicar y destruir el entorno dev; se acepta como riesgo controlado para un entorno académico y efímero.

El pipeline no sustituye a Terraform como fuente de verdad. Cuando se despliega o se hace rollback, el workflow modifica variables como el tag de imagen y reaplica infraestructura. Esto evita cambios manuales directos sobre ECS que quedarían fuera del estado. La operación queda trazada: la task definition activa se deriva de código, variables y estado Terraform.

3.7 Observabilidad mínima

La observabilidad del proyecto se centra en tres elementos: salud, logs correlables y métricas/alertas gestionadas. La salud se expone mediante endpoints `/api/v1/health/live` y `/api/v1/health/ready`. El primero indica que el proceso responde; el segundo comprueba la conectividad con PostgreSQL. Esta distinción permite que el balanceador de carga tome decisiones más precisas sobre si una tarea está lista para recibir tráfico.

Los logs se emiten en formato JSON y se envían a CloudWatch mediante el driver `aws-logs`. El middleware de correlación añade o conserva el header `X-Correlation-ID`, lo devuelve al cliente y lo usa como `TraceIdentifier`. De esta forma, un error observado desde una respuesta HTTP puede buscarse en CloudWatch con el mismo identificador. Para un entorno pequeño, esta capacidad aporta un valor operativo alto sin introducir una plataforma [APM](#) completa.

El módulo de observabilidad crea dashboard CloudWatch, alarmas sobre ALB/ECS/RDS y filtro métrico para errores de aplicación. Las alarmas usan tratamiento conservador de datos ausentes para reducir ruido cuando el entorno está destruido o con ECS a cero tareas. Esta decisión refleja el carácter efímero de dev: la observabilidad debe existir cuando el entorno está activo, pero no debe generar falsas incidencias cuando se ha destruido intencionadamente.

3.8 Seguridad y coste como restricciones de diseño

La seguridad se aborda desde varias capas. A nivel de aplicación, se usan contraseñas con hash, autenticación JWT, autorización por roles y bootstrap protegido por token externo. A nivel de infraestructura, RDS no es público, ECS se ejecuta en subred privada, el ALB es el único punto de entrada y HTTPS se gestiona mediante ACM. A nivel de configuración, los secretos no se versionan y se consumen desde Secrets Manager o variables externas.

El coste también condiciona la arquitectura. NAT Gateway, ALB, ECS Fargate, RDS y CloudWatch generan gasto mientras el entorno está activo. Por ello, el proyecto adopta una estrategia de entorno efímero: se recrea dev para validar, se captura evidencia y se destruye al cierre. Permanecen únicamente recursos base como dominio, hosted zone y backend remo-

to de Terraform. Esta forma de trabajar permite validar una arquitectura real sin dejar gasto innecesario durante semanas.

La combinación de seguridad y coste produce decisiones concretas. Por ejemplo, se acepta no introducir WAF, autoscaling avanzado ni OpenTelemetry en los sprints finales porque ampliarían complejidad y coste sin ser necesarios para demostrar los objetivos. En cambio, sí se priorizan HTTPS, secretos externos, red privada, logs correlables, alarmas básicas, rollback, restore y runbooks, que aportan evidencia directa sobre la capacidad de operar el sistema.

Metodología y planificación

4.1 Enfoque de trabajo

El proyecto se desarrolla con un enfoque iterativo e incremental. Cada iteración busca producir una versión funcional o verificable del sistema, evitando introducir complejidad que no aporte valor directo a los objetivos del trabajo.

4.2 Fases del proyecto

1. Análisis y diseño del alcance, requisitos, modelo de datos y arquitectura.
2. Desarrollo del backend y de los casos de uso principales.
3. Contenerización y preparación del entorno local reproducible.
4. Definición de la infraestructura cloud mediante Terraform.
5. Automatización de integración y despliegue continuo.
6. Incorporación de observabilidad y medidas básicas de seguridad.
7. Validación funcional y análisis del comportamiento en ejecución.
8. Redacción de memoria, conclusiones y líneas futuras.

4.3 Gestión del alcance

La gestión del alcance se ha basado en una decisión explícita: construir un producto funcionalmente pequeño, pero técnicamente completo. En vez de añadir módulos de negocio avanzados, como optimización de rutas, facturación, geolocalización en tiempo real o panel

web, se priorizó que el backend mínimo pudiera desplegarse y operarse en AWS de forma creíble.

Esta decisión reduce el riesgo de terminar con una aplicación amplia pero superficial. El valor del trabajo se concentra en demostrar el ciclo completo de ingeniería: modelado de dominio, API, persistencia, pruebas, contenedores, infraestructura cloud, seguridad básica, CI/CD, migraciones y validación operativa.

El alcance funcional se fijó alrededor de transportes, vehículos, conductores, usuarios, asignaciones, estados y eventos logísticos. Cualquier funcionalidad fuera de ese flujo se trató como línea futura salvo que fuese necesaria para demostrar el despliegue o la operación.

4.4 Ejecución por sprints

La planificación del tramo cloud se organizó en sprints semanales. Esta estructura permitió separar claramente la construcción funcional del backend, la definición de infraestructura, el primer despliegue real y el endurecimiento operativo posterior.

Sprint 6 se centró en observabilidad y hardening de despliegue. Su objetivo no fue añadir nuevos casos de uso de negocio, sino convertir el entorno dev en un sistema más diagnosticable y operable. El trabajo incluyó correlación de peticiones, logging JSON, dashboard CloudWatch, alarmas, verificación de observabilidad en el workflow de despliegue, circuit breaker de ECS, parametrización de health checks y ajuste del camino DNS/HTTPS en la cuenta AWS actual. El sprint terminó con una validación real en AWS y con el destroy del entorno para controlar costes.

Sprint 7 se planteó como cierre de fiabilidad operativa. Tampoco añadió funcionalidad de negocio; se centró en demostrar que el entorno puede recuperarse de un despliegue fallido, volver a una imagen conocida, validar un restore real de RDS desde snapshot manual y destruir todos los recursos efímeros al finalizar. Este sprint incorporó un workflow manual de rollback, un script PowerShell de restore contra una base de datos temporal, endurecimiento de la imagen Docker, ajustes explícitos de runtime y una nueva ronda de validación real en AWS dev.

Sprint 8 se orientó a cerrar la parte defendible de seguridad, coste y operación. El trabajo consistió en revisar IAM, grupos de seguridad, exposición TLS, secretos y postura de red; documentar qué recursos generan coste y cuáles sobreviven intencionadamente a un destroy; definir un procedimiento completo para recrear dev desde cero; y consolidar runbooks para despliegue, rollback, reinicio, bootstrap, restore, búsqueda por correlation id, respuesta a alarmas y cierre de coste.

4.5 Herramientas utilizadas

Las herramientas principales utilizadas han sido:

- ASP.NET Core y .NET 10 para la implementación de la API REST [13].
- Entity Framework Core como ORM y herramienta de migraciones.
- PostgreSQL como base de datos relacional.
- xUnit y WebApplicationFactory para pruebas automatizadas.
- Docker y Docker Compose para empaquetado y ejecución local reproducible.
- Terraform para definir infraestructura AWS como código.
- AWS ECS Fargate, ECR, RDS, ALB, Route 53, ACM, CloudWatch, Secrets Manager y SSM Parameter Store como servicios cloud principales [4, 5, 7, 8].
- GitHub Actions para integración continua y despliegue manual.
- Postman/Newman para pruebas de humo manuales y automatizables.
- LaTeX para la elaboración de la memoria.

Análisis y requisitos

5.1 Actores

El sistema distingue los siguientes actores:

- Usuario no autenticado: puede consultar los endpoints públicos de salud y realizar login, pero no acceder a operaciones de negocio.
- Operador: usuario autenticado encargado de la operación diaria. Puede gestionar transportes, vehículos, conductores, asignaciones, transiciones de estado y eventos logísticos.
- Administrador: usuario autenticado con permisos completos. Además de las operaciones del operador, puede gestionar usuarios, roles y activación de cuentas.
- Plataforma: conjunto de componentes técnicos que interactúan con la API, como Docker, GitHub Actions, ECS, ALB y comprobaciones de salud.

5.2 Requisitos funcionales

Los requisitos funcionales principales son:

Cuadro 5.1: Requisitos funcionales principales

ID	Descripción
RF-01	Exponer endpoints de salud de proceso y de preparación, diferenciando disponibilidad de la aplicación y conectividad con PostgreSQL.
RF-02	Permitir la creación controlada del primer administrador mediante un endpoint de bootstrap protegido por token externo.

RF-03	Autenticar usuarios mediante login con usuario y contraseña, devolviendo un JWT válido.
RF-04	Autorizar endpoints protegidos mediante roles admin y operator.
RF-05	Gestionar usuarios desde endpoints exclusivos para administradores.
RF-06	Gestionar transportes con creación, consulta, actualización, listado filtrado, paginación y borrado lógico.
RF-07	Gestionar vehículos como recursos asignables, incluyendo validación de matrícula y código interno.
RF-08	Gestionar conductores como recursos asignables, incluyendo licencia, contacto y estado activo.
RF-09	Asignar vehículo y conductor a un transporte planificado mediante una acción explícita.
RF-10	Controlar el ciclo de vida de un transporte mediante transiciones de estado válidas.
RF-11	Registrar y consultar eventos logísticos asociados a un transporte, conservando el usuario que los creó.
RF-12	Devolver errores coherentes para validación, autenticación, autorización, recursos inexistentes y conflictos de negocio.

5.3 Requisitos no funcionales

Los requisitos no funcionales se centran en la calidad operativa:

- Reproducibilidad: el entorno local debe poder levantarse con Docker Compose y configuración externa.
- Mantenibilidad: el código debe conservar una estructura simple, con responsabilidades claras y sin dividir el sistema en proyectos innecesarios.
- Persistencia consistente: PostgreSQL y las migraciones de EF Core son la fuente de verdad del esquema.
- Seguridad básica: contraseñas con hash, JWT externo, roles, secretos fuera del repositorio y red privada para ECS y RDS.

- Despliegue reproducible: los recursos cloud deben definirse en Terraform y almacenarse con estado remoto.
- Observabilidad mínima: la API debe exponer salud, emitir logs a CloudWatch y permitir diagnosticar estado del servicio.
- Validación automática: la solución debe compilarse y probarse mediante comandos locales y flujos CI.
- Operación cloud: el entorno dev debe ejecutarse en AWS con HTTPS y base de datos privada.

5.4 Modelo de dominio

El modelo de dominio se articula alrededor de cinco entidades principales:

- AppUser: representa usuarios autenticables, con nombre de usuario, correo, hash de contraseña, rol, estado activo y campos de auditoría.
- Transport: entidad principal del sistema, con referencia, origen, destino, fechas planificadas y reales, estado y asignación opcional de vehículo y conductor.
- Vehicle: recurso material asignable a transportes, identificado por matrícula y, opcionalmente, código interno.
- Driver: recurso humano asignable, identificado por licencia y datos básicos de contacto.
- ShipmentEvent: evento cronológico vinculado a un transporte y al usuario que lo registra.

Las reglas más relevantes son que un transporte nuevo comienza en estado planned, solo puede pasar a in_transit si tiene vehículo y conductor asignados, y los estados delivered y cancelled son terminales. Las entidades operativas aplican borrado lógico, por lo que las restricciones de unicidad se definen sobre registros activos.

5.5 Casos de uso principales

Los casos de uso principales son:

1. Bootstrap del primer administrador: se configura un token externo, se llama al endpoint de bootstrap y se crea el primer usuario admin si no existe otro administrador activo.

2. Login: un usuario activo obtiene un **JWT** para acceder a endpoints protegidos.
3. Administración de usuarios: un administrador lista usuarios, crea operadores o administradores, cambia roles y activa o desactiva cuentas.
4. Gestión operativa: un operador crea transportes, vehículos y conductores, consulta listados y actualiza datos básicos.
5. Asignación: se asigna conjuntamente un vehículo y un conductor a un transporte planificado.
6. Ejecución del transporte: se cambia el estado del transporte siguiendo transiciones válidas y se registran eventos logísticos.
7. Consulta de trazabilidad: se recupera el historial cronológico de eventos de un transporte.

Diseño de la arquitectura

6.1 Visión general

La arquitectura global se basa en una API REST monolítica, [stateless](#) y contenerizada. El cliente accede al sistema a través de un endpoint público gestionado por un [ALB](#), mientras que la ejecución de la aplicación y la base de datos permanecen en subredes privadas. Esta separación permite exponer únicamente el balanceador de carga y mantener PostgreSQL aislado de Internet.

En local, el sistema se ejecuta con Docker Compose, levantando la API y PostgreSQL. En AWS, la API se publica como imagen Docker en ECR y se ejecuta en ECS Fargate. RDS PostgreSQL actúa como almacén persistente. Route 53 resuelve el dominio `api.dev.transitops.net`, ACM emite el certificado [TLS](#) y el [ALB](#) centraliza el tráfico público. La validación cloud admite un fallback HTTP mediante el [DNS](#) técnico del [ALB](#) cuando se recrea el entorno antes de completar la delegación DNS, pero la configuración objetivo de Terraform activa [HTTPS](#) y redirección de HTTP a [HTTPS](#).

La infraestructura se define mediante Terraform, con módulos reutilizables para red, registro de contenedores, base de datos, configuración, observabilidad y runtime de contenedores. Esta decisión permite reproducir el entorno, revisar los cambios de infraestructura como parte del repositorio y destruir el entorno dev tras las validaciones para controlar el coste.

6.2 Arquitectura lógica del backend

El backend se organiza como una solución .NET deliberadamente simple con dos proyectos: `TransitOps.Api` y `TransitOps.Tests`. Dentro de la API se distinguen responsabilidades por carpetas:

- `Controllers`: puntos de entrada HTTP versionados.

- Contracts: objetos de petición y respuesta expuestos por la API.
- Application: servicios de aplicación que contienen la lógica de casos de uso.
- Domain: entidades y enumeraciones del dominio.
- Infrastructure: persistencia, configuración EF Core y migraciones.
- Common, Errors y Middleware: contrato común de respuestas, errores y tratamiento transversal.

No se ha dividido el backend en múltiples proyectos de dominio, aplicación e infraestructura porque el tamaño del sistema no lo justifica. La separación interna por carpetas ofrece suficiente claridad sin añadir coste accidental.

6.3 Arquitectura de datos

PostgreSQL se utiliza como base de datos principal. El esquema se gestiona mediante migraciones de Entity Framework Core, ubicadas bajo TransitOps.Api/Infrastructure/Persistence/Migrations. Esta carpeta es la fuente de verdad del esquema, tanto para local como para cloud.

Las entidades principales aplican campos de auditoría y borrado lógico mediante `deleted_at`. Por ello, varias restricciones de unicidad se han diseñado para registros activos, permitiendo reutilizar identificadores de negocio cuando un registro anterior ha sido eliminado lógicamente. Este criterio aparece en referencias como matrículas de vehículos, licencias de conductores o referencias de transporte.

Los estados y tipos de evento se almacenan como `smallint` con restricciones `check`. Esta solución evita depender de enumerados nativos de PostgreSQL y simplifica la integración con EF Core, manteniendo a la vez validación a nivel de base de datos.

6.4 Arquitectura cloud en AWS

La arquitectura cloud desplegable en AWS incluye Route 53, ACM, [ALB](#), [ECS](#) Fargate, [ECR](#), [RDS](#) PostgreSQL, Secrets Manager, [SSM](#) Parameter Store y CloudWatch [4, 5, 6, 7, 8, 9, 10, 11].

El entorno dev se despliega en la cuenta AWS 661000947340, región eu-west-1. La red está formada por una [VPC](#) con subredes públicas para [ALB](#) y NAT, subredes privadas de aplicación para ECS y subredes privadas de datos para RDS. Los grupos de seguridad restringen el flujo a: Internet hacia [ALB](#), [ALB](#) hacia ECS y ECS hacia RDS.

La API queda preparada para exponerse como <https://api.dev.transitops.net>. En la cuenta objetivo se dispone del dominio transitops.net y de una hosted zone pública de Route 53;

durante Sprint 6 se corrigió la delegación de nameservers para que la validación DNS de ACM fuese visible públicamente. Terraform crea el certificado, los registros de validación, el alias `api.dev.transitops.net`, el listener HTTPS y la redirección HTTP a HTTPS cuando `enable_https = true`. La base de datos RDS no es pública y solo acepta tráfico PostgreSQL desde el grupo de seguridad de ECS.

ListHostedZonesByName				
transitops.net.	/hostedzone/Z9844787W57HXN9F1JR	False	4	

GetDomainDetail	
ns-1256.awsdns-29.org	
ns-1789.awsdns-31.co.uk	
ns-433.awsdns-54.com	
ns-655.awsdns-17.net	

ListResourceRecordSets				
transitops.net.	NS	None	ns-1789.awsdns-31.co.uk.	
transitops.net.	NS	None	ns-1789.awsdns-31.co.uk.	
api.dev.transitops.net.	A	transitops-dev-alb-588986112.eu-west-1.elb.amazonaws.com.	1 7280 980 1299680 86480	

Name	Type	IPAddress
api.dev.transitops.net	A	52.51.247.152
api.dev.transitops.net	A	52.211.4.224

Figura 6.1: Evidencia DNS para el endpoint de desarrollo.

6.5 Decisiones arquitectónicas

Las decisiones arquitectónicas principales han sido:

- Monolito modular frente a microservicios: el dominio y el tamaño del trabajo no justifican la complejidad de microservicios, mensajería o orquestación avanzada.
- ECS Fargate frente a servidores EC2: Fargate reduce administración de infraestructura y encaja bien con una API stateless contenerizada.
- RDS PostgreSQL frente a PostgreSQL autogestionado: RDS simplifica backups, operación y red privada.
- Terraform frente a creación manual: la infraestructura queda versionada, reproducible y revisable [1].
- Secrets Manager y SSM frente a variables hardcodeadas: los secretos no se almacenan en el repositorio y pueden gestionarse de forma separada.
- Migraciones como tarea ECS puntual frente a migraciones automáticas en producción: el despliegue gana control explícito y evita que cada arranque web modifique el esquema.
- OIDC en GitHub Actions frente a claves AWS persistentes: se reduce el riesgo de credenciales de larga duración.

- Observabilidad mínima con CloudWatch frente a plataformas externas: el sprint prioriza logs JSON, correlación, dashboard y alarmas con servicios ya disponibles en AWS, evitando introducir herramientas adicionales antes de que exista una necesidad real.

Estas decisiones favorecen una arquitectura pequeña pero defendible, orientada a demostrar operación real en cloud.

Diseño detallado del sistema

7.1 Criterios de diseño

El diseño de TransitOps parte de una restricción deliberada: el sistema debe ser pequeño en funcionalidad, pero completo en su ciclo técnico. Esto implica priorizar decisiones que permitan explicar, validar y operar el proyecto de extremo a extremo. No se busca maximizar el número de entidades ni de pantallas, sino construir un backend cuya arquitectura sea comprensible y cuya infraestructura pueda reproducirse.

Los criterios principales fueron simplicidad, trazabilidad, separación de responsabilidades y verificabilidad. La simplicidad evita añadir patrones innecesarios. La trazabilidad permite relacionar requisitos, código, pruebas, infraestructura y evidencias. La separación de responsabilidades reduce el acoplamiento entre entrada HTTP, lógica de aplicación, persistencia y operación. La verificabilidad obliga a que cada decisión pueda comprobarse con tests, comandos o evidencia cloud.

Esta forma de diseñar se refleja en decisiones concretas: una única API monolítica, una base PostgreSQL, una imagen Docker, un entorno dev reproducible, Terraform como fuente de verdad y CloudWatch como punto mínimo de observabilidad. Cada pieza cumple una función clara y evita introducir componentes que no serían defendibles dentro del alcance del TFG.

7.2 Diseño del contrato HTTP

La API se organiza bajo la versión `/api/v1`. Versionar la ruta desde el inicio permite evolucionar el contrato sin romper clientes futuros, aunque el proyecto no implemente múltiples versiones. Los endpoints de negocio usan recursos claros: autenticación, usuarios, transportes, vehículos, conductores y eventos. Las operaciones siguen una semántica HTTP convencional: GET para consulta, POST para creación o acciones, PUT para actualización y DELETE para borrado lógico.

El contrato de respuesta busca homogeneidad. Los errores se devuelven con códigos HTTP adecuados y un cuerpo común que evita exponer detalles internos. Esta decisión mejora la experiencia de consumo y facilita los tests. Por ejemplo, una petición no autenticada debe terminar en 401; un usuario sin permisos, en 403; un recurso inexistente, en 404; y un conflicto de negocio, en 409.

La correlación de peticiones forma parte del contrato operativo. Todas las respuestas incluyen X-Correlation-ID. Si el cliente envía ese header, la API conserva el valor; si no lo envía, genera uno. Este comportamiento permite que un cliente o una prueba de humo etiquete una petición concreta y la busque después en CloudWatch. Aunque no cambia la funcionalidad de negocio, sí mejora la capacidad de diagnóstico del sistema desplegado.

Los endpoints de salud se diseñan con una finalidad operativa. `/api/v1/health/live` valida que el proceso responde, mientras que `/api/v1/health/ready` comprueba la conexión con base de datos. El ALB usa readiness porque una API viva pero sin PostgreSQL no debería recibir tráfico real. Esta distinción es sencilla, pero importante para evitar falsos positivos en despliegue.

7.3 Diseño del dominio

El dominio se centra en la operación básica de transportes. La entidad `Transport` representa el elemento principal y contiene referencia, origen, destino, fechas, estado y asignaciones opcionales. El vehículo y el conductor se modelan como recursos independientes porque pueden existir antes de estar asignados y porque sus datos deben gestionarse de forma separada. Los eventos logísticos proporcionan trazabilidad temporal.

El ciclo de vida de un transporte se controla mediante estados. Un transporte comienza en estado planificado. Para pasar a en tránsito debe tener recursos asignados. Una vez entregado o cancelado, entra en un estado terminal. Esta regla evita secuencias incoherentes, como entregar un transporte no iniciado o modificar el estado después de una cancelación.

El borrado lógico se usa para entidades operativas. En lugar de eliminar físicamente transportes, vehículos o conductores, se marca una fecha de borrado. Esto permite conservar histórico y evita romper relaciones pasadas. Al mismo tiempo, los índices únicos se diseñan para registros activos, de modo que una referencia o matrícula pueda reutilizarse si el registro anterior se ha retirado lógicamente.

Los usuarios se separan por rol. El rol `admin` puede gestionar usuarios y configuración básica de acceso; el rol `operator` puede ejecutar operaciones logísticas. Esta división es suficiente para el alcance actual y permite demostrar autorización sin introducir una matriz de permisos compleja.

7.4 Diseño de persistencia

El esquema relacional se deriva de las entidades principales. La tabla de usuarios almacena credenciales hash, rol y estado activo. La tabla de transportes mantiene la información operativa y claves foráneas opcionales hacia vehículo y conductor. Las tablas de vehículos y conductores mantienen sus identificadores propios. La tabla de eventos referencia transporte y usuario creador, preservando quién registró cada hito.

Las migraciones de EF Core actúan como registro de evolución del esquema. Esta elección permite que el repositorio contenga no solo el modelo actual, sino también la historia de cambios. En un entorno cloud, las migraciones se ejecutan mediante una tarea ECS puntual para que el cambio de base de datos sea un paso visible del despliegue.

La persistencia debe soportar dos escenarios: ejecución local y ejecución cloud. En local, la cadena de conexión apunta al servicio PostgreSQL definido en Docker Compose. En AWS, la cadena se obtiene desde Secrets Manager y apunta a RDS. La aplicación no necesita cambiar su código entre ambos escenarios; lo que cambia es la configuración externa.

El diseño también considera restore. La prueba de Sprint 7 demostró que una base restaurada desde snapshot puede recibir la tarea de migración. Esto es relevante porque la restauración no se valida únicamente viendo que RDS crea una instancia, sino comprobando que la aplicación puede usarla para ejecutar su procedimiento operativo más delicado.

7.5 Diseño de autenticación y autorización

El sistema necesita un primer administrador para empezar a operar, pero no puede depender de crear usuarios manualmente en base de datos. Por ello se implementa un endpoint de bootstrap protegido por token externo. El token se configura fuera del repositorio y el endpoint solo crea el primer administrador si no existe ya otro activo. Esta regla evita que el bootstrap se convierta en una puerta permanente para crear administradores arbitrarios.

El login valida usuario y contraseña. Las contraseñas no se almacenan en claro; se guardan como hash. Si las credenciales son válidas, la API emite un JWT con identidad y rol. Los endpoints protegidos usan ese token para identificar al usuario y aplicar autorización. La duración, emisor, audiencia y clave de firma se configuran externamente.

La autorización se implementa con roles simples. Esta elección evita un sistema de permisos sobredimensionado. En el alcance del TFG basta distinguir administración y operación. Si el proyecto evolucionase hacia producción, se podría añadir una matriz de permisos más fina, auditoría de acciones administrativas o integración con un proveedor externo de identidad.

El diseño de errores evita revelar información sensible. Un login fallido no debe indicar si el usuario existe o si solo falló la contraseña. Los errores internos se registran con correlation id,

pero al cliente se devuelve un mensaje controlado. Esta separación entre diagnóstico interno y respuesta externa es una medida básica de seguridad.

7.6 Diseño de configuración

La configuración se organiza por niveles. Los valores no sensibles pueden estar en app-settings, variables de entorno o SSM Parameter Store. Los valores sensibles, como cadena de conexión, clave JWT y token de bootstrap, se almacenan fuera del repositorio. En local se pueden proporcionar mediante .env o secretos de desarrollo. En AWS se consumen desde Secrets Manager.

Esta separación evita que el código fuente contenga credenciales. También permite recrear el entorno sin modificar la imagen Docker. La misma imagen puede ejecutarse en local o en cloud porque obtiene la configuración desde el entorno. Esto encaja con el principio de construir una vez y desplegar con configuración externa.

Terraform crea los secretos como contenedores de configuración, pero no versiona sus valores reales. Esta diferencia es importante: el repositorio puede conocer el nombre del secreto que la task definition debe leer, pero no debe contener el password de base de datos ni la clave JWT. Los valores se cargan durante el despliegue o mediante scripts controlados.

Durante los sprints se detectó un caso práctico relacionado con Secrets Manager: algunos secretos quedaron programados para eliminación tras un ciclo de destroy. La recuperación exigió restaurarlos o forzar su eliminación antes de recrearlos. Esta experiencia se documentó porque forma parte del comportamiento real de AWS y afecta a la operación del entorno dev.

7.7 Diseño de despliegue

El despliegue se divide en fases. Primero se crea la infraestructura con ECS a cero tareas. Esto permite construir red, ALB, ECR, RDS, secretos y observabilidad sin intentar arrancar una imagen que todavía no existe. Después se publica la imagen en ECR, se cargan secretos de runtime, se ejecutan migraciones y se escala ECS a una tarea.

Esta secuencia evita un problema habitual en despliegues iniciales: crear un servicio que intenta arrancar antes de tener imagen o configuración. Al separar infraestructura base, publicación de artefacto y activación de servicio, cada error se diagnostica de forma más clara. Si falla la migración, el problema está en base de datos o configuración; si falla ECS, puede revisarse task definition, imagen o secretos; si falla health, se revisan ALB, target group y readiness.

El target group del ALB usa la ruta de readiness. El periodo de gracia de ECS evita sustituir tareas demasiado pronto durante el arranque. El deployment circuit breaker permite que ECS

marque como fallido un despliegue que no consigue estabilizarse. En Sprint 7 se probó un tag inexistente para demostrar este comportamiento y luego se recuperó aplicando de nuevo una imagen buena.

DNS y HTTPS se gestionan con Route 53 y ACM. Terraform solicita el certificado, crea registros de validación y configura listeners del ALB. La incidencia de Sprint 6 con nameservers permitió validar una parte importante del proceso: no basta con tener una hosted zone; el dominio registrado debe delegar en los nameservers correctos para que ACM pueda validar públicamente el CNAME.

7.8 Diseño de validación

La validación del proyecto se diseñó en capas. La primera capa son tests automatizados de backend, que comprueban reglas funcionales y contratos HTTP. La segunda capa son validaciones locales con Docker, que verifican que la aplicación puede ejecutarse con PostgreSQL en contenedor. La tercera capa es Terraform, que valida sintaxis, formato, plan y aplicación de infraestructura. La cuarta capa es AWS real, donde se comprueba que el sistema responde por HTTPS, ejecuta migraciones, registra logs y puede destruirse.

Este enfoque evita depender de una única prueba final. Un despliegue cloud correcto no sustituye a los tests de dominio; un test local correcto no prueba Route 53 o ACM; un terraform validate correcto no demuestra que ECS arranque. Cada capa cubre un tipo de riesgo diferente.

La validación cloud se orienta a evidencias observables: health 200, login 200, bootstrap 201 o 409, logs JSON con correlation id, dashboard creado, alarmas presentes, task definition activa, RDS privado y destroy limpio. Estos resultados se registran en documentación para que la defensa no dependa de recordar comandos ejecutados.

7.9 Decisiones descartadas

Durante el diseño se descartaron varias alternativas. La primera fue construir un frontend completo. Aunque habría hecho el proyecto más visible, también habría desplazado tiempo desde cloud, Terraform, observabilidad y operación. Dado el objetivo del TFG, se consideró más valioso profundizar en backend y despliegue.

También se descartó dividir el sistema en microservicios. El dominio no lo justifica y habría introducido complejidad en red, despliegue, observabilidad y consistencia de datos. Un monolito modular resulta más adecuado para demostrar ingeniería de extremo a extremo sin multiplicar componentes.

No se introdujo Kubernetes. ECS Fargate cubre la necesidad de ejecutar contenedores gestionados con menor carga operativa. Kubernetes habría requerido justificar clúster, ingress,

controladores, despliegues y observabilidad adicional. Para el alcance del proyecto, esa complejidad no aporta una ventaja proporcional.

Tampoco se añadió OpenTelemetry/X-Ray en los sprints finales. La correlación mediante header y logs JSON ofrece un nivel suficiente para diagnosticar peticiones en un entorno pequeño. La trazabilidad distribuida queda como evolución natural si el sistema creciese o si apareciesen múltiples servicios.

Implementación del backend

8.1 Estructura del proyecto

La solución contiene dos proyectos principales. TransitOps.Api implementa el servicio HTTP, la lógica de aplicación, el dominio y la persistencia. TransitOps.Tests agrupa las pruebas automatizadas. Esta estructura mantiene bajo el número de proyectos y facilita navegar el código durante el desarrollo y la defensa.

ASP.NET Core se utiliza como framework HTTP [13]. La API define controladores versionados bajo /api/v1, un contrato común de respuesta y middleware de errores para homogeneizar las salidas. La configuración se obtiene desde appsettings, variables de entorno, secretos locales en desarrollo y servicios AWS en despliegue.

8.2 API REST

La API REST cubre las siguientes áreas:

- Salud: GET /api/v1/health/live y GET /api/v1/health/ready.
- Autenticación: POST /api/v1/auth/bootstrap-admin y POST /api/v1/auth/login.
- Usuarios: listado, detalle, creación, cambio de rol y activación/desactivación.
- Transportes: creación, listado filtrado y paginado, detalle, actualización, borrado lógico, asignación y cambio de estado.
- Vehículos y conductores: CRUD con borrado lógico.
- Eventos logísticos: creación y consulta cronológica por transporte.

Las respuestas de éxito se encapsulan en un contrato común. Los errores distinguen validación, autenticación, autorización, no encontrado y conflicto de negocio, usando códigos HTTP coherentes como 400, 401, 403, 404 y 409.

8.3 Persistencia

La persistencia se implementa con Entity Framework Core sobre PostgreSQL. El `TransitOpsDbContext` representa el punto central de acceso a datos y define entidades, relaciones, restricciones e índices. Las migraciones permiten evolucionar el esquema de forma controlada.

El proyecto evolucionó desde una primera aproximación basada en SQL de arranque hacia un modelo donde EF Core migrations es la fuente canónica. Esto simplifica el despliegue y evita divergencias entre entorno local, pruebas y AWS.

En local Docker, la API puede aplicar migraciones al arrancar cuando la opción `Database: ApplyMigrationsOnStartup` está habilitada. En AWS, en cambio, las migraciones se ejecutan mediante el modo `--migrate-only`, como tarea ECS puntual.

8.4 Autenticación y autorización

El sistema incorpora un flujo controlado para crear el primer administrador. El endpoint `POST /api/v1/auth/bootstrap-admin` requiere un token externo configurado fuera del repositorio y solo permite crear el primer usuario administrador si no existe ya un administrador activo.

El login se realiza mediante `POST /api/v1/auth/login`. Las contraseñas se almacenan como hash y, si las credenciales son válidas, la API devuelve un [JWT](#). Este token incluye la identidad y el rol del usuario, permitiendo proteger los endpoints de negocio.

El modelo de autorización distingue admin y operator. El administrador puede gestionar usuarios; el operador se limita a las operaciones de transporte. Además, la API impide dejar el sistema sin ningún administrador activo.

8.5 Pruebas del backend

Las pruebas del backend se encuentran en `TransitOps.Tests`. Cubren endpoints de salud, transportes, vehículos, conductores, autenticación, usuarios, reglas de estado y flujos de negocio principales. Se utiliza `WebApplicationFactory` para ejecutar pruebas de integración sobre la API.

En el estado actual, la suite principal ejecuta 84 pruebas automatizadas. Estas pruebas se complementan con colecciones Postman/Newman para verificar flujos contra una API viva y con pruebas de humo sobre el entorno AWS desplegado.

Infraestructura y despliegue

9.1 Contenerización

La API se empaqueta en una imagen Docker construida a partir de su Dockerfile. El contenedor expone únicamente el puerto 8080, instala las dependencias necesarias para el runtime .NET y se configura mediante variables de entorno. La imagen se utiliza tanto para despliegue cloud como para validar el comportamiento de empaquetado.

En Sprint 7 se añadió un fichero .dockerignore para reducir el contexto de construcción. Se excluyen artefactos de compilación bin/ y obj/, metadatos .git/, directorios .terraform/, ficheros locales con secretos, documentación y artefactos pesados de la memoria. Esto acelera la construcción, evita transferir material innecesario al daemon Docker y reduce el riesgo de empaquetar accidentalmente información que no forma parte del runtime.

El entorno local se reproduce con Docker Compose, levantando la API y PostgreSQL. La configuración sensible se proporciona mediante un fichero .env no versionado. Esto permite ejecutar el sistema local sin instalar PostgreSQL de forma manual y mantiene una ruta de validación cercana al despliegue real.

9.2 Infraestructura como código

Terraform se organiza bajo infra/terraform. La estructura separa un bootstrap de estado remoto, módulos reutilizables y raíces de entorno:

- bootstrap/remote_state: crea el bucket S3 de estado y la tabla DynamoDB de bloqueo.
- modules/platform_foundation: VPC, subredes, rutas, NAT y grupos de seguridad.
- modules/container_registry: repositorio ECR.
- modules/database: RDS PostgreSQL.

- `modules/runtime_config`: Secrets Manager y SSM.
- `modules/container_runtime`: ECS, task definition, ALB, listeners, Route 53 y ACM.
- `modules/observability`: grupo de logs, dashboard CloudWatch, alarmas y notificación SNS opcional.
- `modules/github_oidc`: rol de despliegue para GitHub Actions.
- `environments/dev`: composición real del entorno de desarrollo.

El estado remoto se almacena en S3 y se bloquea con DynamoDB. Esto evita estados locales divergentes y reduce el riesgo de cambios concurrentes.

9.3 Entorno AWS

El entorno desplegado utiliza:

- Cuenta AWS: 661000947340.
- Región: eu-west-1.
- Entorno: dev.
- Dominio objetivo validado: `api.dev.transitops.net`.
- Hosted zone: `transitops.net` en Route 53.
- Identificador de hosted zone: `Z0844787W37HXN9FIJR`.
- Fallback operativo: DNS público del ALB cuando el entorno se recrea antes de completar DNS/ACM.

Los recursos siguen la convención `transitops-dev-<componente>` y se etiquetan con proyecto, entorno, propietario, repositorio, servicio, origen Terraform y las etiquetas `TerraformStack` y `ResourceGroup`. Esta nomenclatura permite identificar recursos y costes, y comprobar tras un `terraform destroy` que no quedan recursos del entorno dev.

Como el entorno dev es educativo y desechable, la configuración actual desactiva la retención automática de backups de RDS y omite snapshots finales. Esta decisión reduce costes residuales en ciclos frecuentes de creación y destrucción, sin impedir que un entorno futuro de producción defina una política de backup más conservadora.

9.4 Despliegue de la aplicación

El flujo de despliegue aplicado en dev durante la primera validación cloud fue:

1. Crear la infraestructura base con ECS en `desired_count = 0`, evitando arrancar tareas antes de tener imagen en ECR.
2. Construir la imagen Docker de la API.
3. Publicar la imagen en ECR con el tag `sprint4-local-d00aa6b`.
4. Actualizar la task definition para usar la imagen publicada.
5. Cargar los secretos de runtime en Secrets Manager.
6. Ejecutar migraciones con una tarea ECS puntual usando `--migrate-only`.
7. Escalar el servicio ECS a `desired_count = 1`.
8. Activar Route 53, ACM, HTTPS y redirección HTTP a HTTPS.
9. Validar el endpoint público de readiness sobre HTTPS.

En Sprint 6 se repitió la recreación del entorno en la cuenta AWS actual 661000947340. La imagen validada fue `sprint6-local-6bb910c`; se restauraron e importaron secretos que habían quedado programados para eliminación tras un ciclo anterior, se ejecutó una tarea ECS de migración con código de salida 0, se escaló el servicio a `desired = 1` y se validaron readiness, bootstrap de administrador y login. La validación funcional principal de Sprint 6 se hizo mediante el DNS HTTP del ALB, y posteriormente se corrigió la delegación DNS de `transitops.net` en la misma cuenta para que el siguiente terraform apply pueda converger directamente con `enable_https = true`.

El módulo de runtime incluye además el deployment circuit breaker de ECS con rollback, periodo de gracia para health checks y parámetros explícitos para la comprobación del target group: ruta `/api/v1/health/ready`, intervalo, timeout y umbrales de healthy/unhealthy. Esto reduce el riesgo de dejar una versión fallida recibiendo tráfico.

Sprint 7 completó este hardening con un `stopTimeout` explícito de 30 segundos en la task definition, un `deregistration_delay` corto en dev, y cadenas de conexión cloud con timeouts y pooling configurados de forma explícita. El objetivo no es optimizar rendimiento en abstracto, sino hacer más predecible el arranque, la parada y la ejecución puntual de migraciones en ECS. La validación real usó la imagen `sprint7-good-3dca035`, ejecutó migraciones con exit code 0 y dejó temporalmente el servicio en `desired = 1` y `running = 1` antes del cierre de coste.

9.5 Rollback y restore

La recuperación ante despliegues fallidos se resolvió de forma coherente con el resto del proyecto: Terraform sigue siendo la fuente de verdad. El rollback manual se ejecuta reaplicando el entorno con un tag de imagen conocido, por ejemplo `sprint7-good-3dca035`, y manteniendo `ecs_desired_count = 1`. El workflow `rollback-dev.yml` automatiza esta operación y comprueba después que ECS queda estable y que el health check responde correctamente.

También se ejecutó una prueba controlada de fallo usando el tag inexistente `sprint7-missing-3dca035`. Este escenario forzó un error de despliegue por imagen no disponible, permitió observar los eventos de ECS y demostró el comportamiento del deployment circuit breaker. La tarea falló con `CannotPullContainerError`. Tras la prueba se reaplicó el tag bueno `sprint7-good-3dca035` para que el estado de Terraform y la task definition volvieran a quedar alineados, y el readiness respondió de nuevo 200.

Para la base de datos, dev mantiene `database_backup_retention_days = 0` y no genera snapshots finales al destruirse. La fiabilidad se valida mediante snapshot manual: el script `scripts/cloud/aws/Invoke-RdsRestoreTest.ps1` crea un snapshot de la base principal, restaura una instancia temporal, crea un secreto temporal con la cadena de conexión restaurada, registra una task definition temporal y ejecuta `--migrate-only` contra esa base. El criterio de éxito es código de salida 0. Salvo que se active una opción para conservar evidencia, el propio script elimina la base restaurada, el secreto, la task definition temporal, la política IAM temporal y el snapshot manual.

```
[
  "2026-05-24T21:43:29.775000+02:00",
  "(service transitops-dev-api-svc) (deployment ecs-svc/5443249335865733949) deployment failed: tasks failed to start."
],
```

```
Restore migrate-only task exit code: 0
KeepEvidence set; temporary restore resources were intentionally preserved.
```

```
-----
| DescribeDBSnapshots |
+-----+
| transitops-dev-db-sprint7-restore-test |
| available |
+-----+
-----
```

```
-----
| DescribeDBInstances |
+-----+
| transitops-dev-db-restore-sprint7 |
| available |
+-----+
-----
```

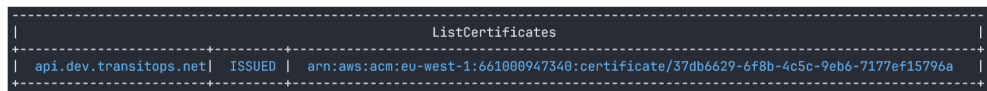
Figura 9.1: Evidencia de rollback automático y restore real de RDS.

9.6 DNS, certificados y HTTPS

El endpoint objetivo es `https://api.dev.transitops.net`. Para lograrlo, Terraform solicita un certificado público en ACM para `api.dev.transitops.net`, crea el CNAME de validación en Route

53, espera a que el certificado pase a estado ISSUED, crea el listener HTTPS 443 en el ALB y convierte el listener HTTP 80 en una redirección permanente hacia HTTPS.

Durante la activación en la cuenta 661000947340 apareció una incidencia real de DNS: el dominio registrado transitops.net apuntaba a nameservers distintos de los de la hosted zone usada por Terraform. ACM permanecía en PENDING_VALIDATION aunque el CNAME existía en Route 53, porque el DNS público no consultaba esa zona. La corrección consistió en actualizar los nameservers del dominio registrado para que coincidieran con la hosted zone Z0844787W37HXN9FIJR. Tras la propagación, el CNAME de validación fue visible desde DNS público y el certificado pasó a ISSUED.



ListCertificates		
api.dev.transitops.net	ISSUED	arn:aws:acm:eu-west-1:661000947340:certificate/37db6629-6f8b-4c5c-9eb6-7177ef15796a

Figura 9.2: Evidencia del certificado ACM emitido para el endpoint de desarrollo.

9.7 Cierre del entorno y control de costes

Después de capturar la evidencia de Sprint 6, el entorno dev se destruyó con terraform destroy para evitar gasto evitable. La destrucción eliminó ALB, listeners, certificado ACM, registros DNS de api.dev.transitops.net, ECS, ECR, RDS, NAT Gateway, VPC, subredes, grupos de seguridad, CloudWatch logs, dashboard, alarmas y secretos del entorno. El repositorio ECR necesitó una limpieza previa de imágenes, ya que AWS no permite eliminar un repositorio no vacío.

En Sprint 7 se ajustó el módulo de ECR para permitir force_delete en dev. Esta decisión evita que una imagen publicada para validar rollback bloquee el destroy final. El ajuste es deliberadamente parametrizable para que un entorno productivo pueda conservar una política más restrictiva.

Tras el destroy de Sprint 7, Terraform informó 72 destroyed. Se verificó que el state quedaba vacío y que no existían recursos costosos asociados al stack transitops-dev: ALB, RDS, ECS, ECR, certificado ACM, registros DNS del subdominio, log groups, dashboards, alarmas, snapshots temporales ni secretos. El NAT Gateway quedó en estado deleted. Permanecen fuera del destroy el dominio registrado transitops.net, la hosted zone pública y los recursos de backend remoto de Terraform, que son recursos base de bajo coste y necesarios para recrear el entorno.

9.8 Gestión de configuración y secretos

La configuración sensible se externaliza. La cadena de conexión a PostgreSQL, la clave de firma JWT y el token de bootstrap se almacenan en Secrets Manager. Otros parámetros, como emisor, audiencia y expiración del JWT, se almacenan en SSM Parameter Store o se proporcionan como variables no secretas.

En el repositorio solo se incluyen ejemplos y nombres de variables. Los ficheros locales que pueden contener secretos, como `terraform.tfvars`, `backend.hcl` o `.env`, están excluidos del control de versiones.

Integración y despliegue continuo

10.1 Objetivos del pipeline

El objetivo del pipeline es convertir el despliegue en un proceso repetible y auditable. Antes de desplegar, el sistema debe restaurar dependencias, compilar, ejecutar pruebas, construir la imagen Docker y validar Terraform. En el despliegue cloud, además, debe publicar la imagen en ECR, aplicar infraestructura, ejecutar migraciones y realizar pruebas de humo.

El pipeline se diseña para ejecución manual en dev. Esta decisión evita despliegues accidentales durante el desarrollo del TFG y permite capturar evidencias controladas.

10.2 Validación automática

La validación automática incluye:

- dotnet restore para restaurar dependencias.
- dotnet build para compilar la solución.
- dotnet test para ejecutar pruebas automatizadas.
- Validación de migraciones EF Core en el flujo de CI base.
- docker compose config para comprobar que la composición local es válida.
- terraform fmt y terraform validate para asegurar formato y coherencia de infraestructura.
- Validación de recursos de observabilidad tras el despliegue: log group, dashboard CloudWatch, alarmas y presencia de logs correlados.

10.3 Publicación de artefactos

La imagen de la API se construye a partir de `TransitOps.Api/Dockerfile`. En el flujo de despliegue, la imagen se etiqueta con el identificador del commit o con un tag operativo, y se publica en el repositorio ECR `transitops/api`. En el despliegue real de Sprint 4 se publicó la imagen `sprint4-local-d00aa6b`. En la validación de Sprint 6 se publicó y desplegó la imagen `sprint6-local-6bb910c`.

ECR actúa como frontera entre construcción y ejecución: ECS no arranca una versión nueva de la API hasta que la task definition apunta a una imagen disponible.

10.4 Despliegue automatizado

Se han definido dos workflows manuales:

- `terraform-dev.yml`: inicializa Terraform, valida formato y configuración, genera plan y permite aplicar manualmente.
- `deploy-dev.yml`: ejecuta build, tests, construcción de imagen, publicación en ECR, Terraform, migraciones ECS, escalado del servicio, health check, bootstrap de administrador, login y verificaciones mínimas de observabilidad.
- `rollback-dev.yml`: reaplica Terraform con un tag de imagen conocido, espera estabilidad de ECS, comprueba que la task definition apunta a esa imagen y ejecuta un smoke test de readiness.

GitHub Actions accede a AWS mediante OIDC y asume un rol IAM de despliegue específico para TransitOps:

`transitops-dev-github-actions-deploy-role`

No se utilizan claves AWS persistentes en GitHub. El rol mantiene permisos amplios para operar Terraform sobre dev, pero su trust policy limita la asunción al repositorio y rama configurados.

En Sprint 6 se amplió el workflow de despliegue para consumir variables de entorno relacionadas con observabilidad y DNS. La variable `ALARM_EMAIL`, si existe en GitHub, se pasa a Terraform como `TF_VAR_alarm_email` para crear una suscripción SNS de email. Cuando no se configura, Terraform crea las alarmas sin acciones de notificación, evitando depender de una dirección personal para que el entorno sea reproducible.

Después de aplicar infraestructura, el workflow captura outputs relevantes del módulo de observabilidad y comprueba que existen el grupo de logs, el dashboard y las alarmas esperadas.

También busca logs JSON con un identificador de correlación generado durante el smoke test. Estas comprobaciones convierten la observabilidad en un criterio de despliegue, no en una revisión manual posterior.

En Sprint 7 se añadió un flujo de rollback explícito. La operación no reconstruye la aplicación ni cambia el contrato HTTP: recibe un `api_image_tag` ya publicado en ECR y lo convierte en el tag deseado de la task definition mediante Terraform. Esto deja el rollback trazable en el state, evita cambios manuales sobre ECS y permite comprobar que la imagen activa coincide con el tag esperado.

En Sprint 8 se cerró el camino de recreación completa del entorno dev. El procedimiento parte de un state vacío tras `terraform destroy`, aplica la plataforma con ECS a cero tareas, publica una imagen con tag operativo, carga secretos de runtime, ejecuta migraciones con `--migrate-only`, escala ECS a una tarea y valida HTTPS, login y observabilidad. La ejecución real usó el tag `sprint8-good-fdf98b9` y una tarea de migración finalizada con exit code 0. Este flujo convierte la reconstrucción del entorno en una operación documentada, no en una secuencia de pasos recordados de forma informal.

```

DescribeImages
+-----+-----+-----+
| 2026-05-24T21:58:36.789000+02:00 | None | sha256:2f49cc3c19f34f47fda031b78b0825c006633cad2608594b72928c36b4c30912 |
| 2026-05-24T21:58:36.789000+02:00 | None | sha256:77d8820ec39791f8c8d78ba41b56ca77fd2a2c8c5b22197da316b082cee191c3 |
| 2026-05-24T21:58:37.156000+02:00 | sprint9-final-3b13415 | sha256:a688fac65d2dc3acf26fa228732d77368ab382fc46a185f642084c618aae9c5 |
+-----+-----+-----+

DescribeServices
+-----+
| transitops-dev-api-svc |
| ACTIVE                |
| 1                      |
| 1                      |
| arn:aws:ecs:eu-west-1:661000947340:task-definition/transitops-dev-api:11 |
+-----+

PS C:\Users\pey\Projects\TransitOps> curl.exe -i "$endpoint/api/v1/health/ready"
HTTP/1.1 200 OK
Date: Sun, 24 May 2026 20:40:56 GMT
Content-Type: application/json; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
Server: Kestrel
X-Correlation-ID: 21708f4b2bd149829ca9e3e851045ba2
{"data":{"status":"ready","service":"TransitOps.Api","environment":"Production","databaseConnectionAvailable":true},
92982","pagination":null}}

```

Figura 10.1: Evidencia de despliegue cloud final: imagen publicada en ECR, servicio ECS estable y readiness HTTPS con respuesta 200.

```

PS C:\Users\pey\Projects\TransitOps\infra\terraform\environments\dev> aws ecs describe-task-definition `
> --task-definition transitops-dev-api `
> --region eu-west-1 `
> --profile aws-pey-v1 `
> --query "taskDefinition.containerDefinitions[0].image" `
> --output text
661000947340.dkr.ecr.eu-west-1.amazonaws.com/transitops/api:sprint9-final-3b13415

```

Figura 10.2: Verificación de la task definition activa apuntando al tag operativo validado.

```
PS C:\Users\pey\Projects\TransitOps\infra\terraform\environments\dev> terraform output -json | ConvertFrom-Json |
>>   Select-Object -ExpandProperty container_runtime |
>>   Select-Object -ExpandProperty value
{
  alb_arn_suffix      : app/transitops-dev-alb/2cb9f1cf7ee30c6a
  alb_dns_name        : transitops-dev-alb-580986112.eu-west-1.elb.amazonaws.com
  api_dns_record_fqdn : api.dev.transitops.net
  cluster_name        : transitops-dev-cluster
  http_listener_arn    : arn:aws:elasticloadbalancing:eu-west-1:661000947340:listener/app/transitops-dev-alb/2cb9f1cf7ee30c6a/1bf5036d1ce2f62e
  https_listener_arn   : arn:aws:elasticloadbalancing:eu-west-1:661000947340:listener/app/transitops-dev-alb/2cb9f1cf7ee30c6a/22bdf9c2786b0da1
  service_name         : transitops-dev-api-svc
  target_group_arn     : arn:aws:elasticloadbalancing:eu-west-1:661000947340:targetgroup/transitops-dev-api-tg/a3c5f2732bd4dbb0
  target_group_arn_suffix : targetgroup/transitops-dev-api-tg/a3c5f2732bd4dbb0
  task_definition_arn  : arn:aws:ecs:eu-west-1:661000947340:task-definition/transitops-dev-api:11
}
```

Figura 10.3: Outputs de Terraform tras la recreación del entorno dev, incluyendo ALB, DNS, ECS y task definition activa.

Observabilidad y seguridad

11.1 Comprobaciones de salud

La API expone dos endpoints de salud. `/api/v1/health/live` indica que el proceso está vivo. `/api/v1/health/ready` comprueba la conectividad real con PostgreSQL mediante el `DbContext`. Esta distinción permite diferenciar entre una aplicación arrancada y una aplicación preparada para servir tráfico.

En AWS, el target group del Application Load Balancer utiliza la ruta de readiness para decidir si la tarea ECS puede recibir tráfico. También se ha usado el mismo endpoint como prueba de humo final. En Sprint 6 se fijó esta ruta como valor por defecto parametrizable del target group y se añadió un periodo de gracia de ECS para evitar que tareas recién arrancadas sean sustituidas antes de completar el arranque.

11.2 Logs y métricas

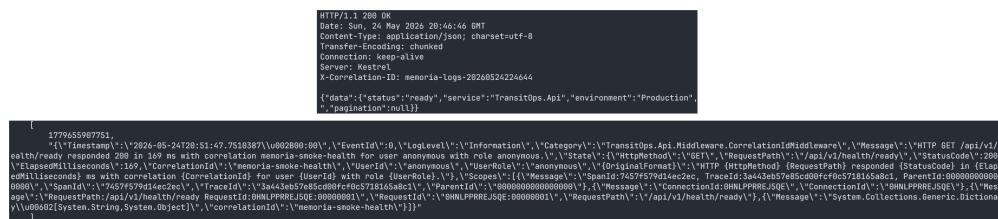
Los logs del contenedor se envían a CloudWatch Logs mediante el driver `awslogs` de ECS. El grupo de logs sigue la convención `/aws/ecs/transitops/dev/api`. Esta integración permite inspeccionar errores de arranque, fallos de configuración, problemas de conexión con RDS y salidas de tareas puntuales como las migraciones.

En Sprint 6 se sustituyó la salida de consola no estructurada por logging JSON mediante `AddJsonConsole`. La configuración incluye scopes, marcas temporales UTC y campos que CloudWatch puede filtrar. También se añadió un middleware de correlación de peticiones:

- acepta X-Correlation-ID si el cliente lo proporciona;
- genera un identificador nuevo si el header no existe;
- devuelve siempre X-Correlation-ID en la respuesta;

- alinea `HttpContext.TraceIdentifier` con ese valor;
- abre un scope de logging con `CorrelationId`;
- registra por petición método, ruta, código HTTP, duración, usuario y rol cuando existen.

Esto permite seguir una petición concreta desde la respuesta HTTP hasta los logs de CloudWatch. Además, los errores gestionados por el middleware de excepciones heredan el mismo identificador, por lo que el campo de error devuelto al cliente y el log del servidor quedan correlados.



```

HTTP/1.1 200 OK
Date: Sun, 24 May 2020 20:46:40 GMT
Content-Type: application/json; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
Server: Kestrel
X-Correlation-ID: memoria-logs-20268524224644

{"data":{"status":"ready","service":"TransitOps.Api","environment":"Production","pagination":null}}

{"@timestamp":"2020-05-24T20:51:47.7518387Z","@version":1,"EventId":8,"LogLevel":"Information","Category":"TransitOps.Api.Middleware.CorrelationIdMiddleware","Message":"HTTP GET /api/v1/health/ready responded 200 in 169 ms with correlation memoria-smoke-health for user anonymous with role anonymous","State":{"HttpRequest":{"Method":"GET","RequestPath":"/api/v1/health/ready"},"StatusCode":200,"ElapsedMilliseconds":169,"CorrelationId":"memoria-smoke-health","UserId":"anonymous","UserRole":"anonymous","OriginalFormat":"HTTP (HttpRequest) responded (StatusCode) in (ElapsedMilliseconds) ms with correlation (CorrelationId) for user (UserId) with role (UserRole)."},"Scopes":[{"Message":"SpanId:7457f579d14c2ec2, TraceId:3a43ab57e85cd00fc0c573165a8c1, ParentId:0000000000000000","SpanId":"7457f579d14c2ec2","TraceId":"3a43ab57e85cd00fc0c573165a8c1","ParentId":"0000000000000000","Message":"ConnectionId:0000000000000000","ConnectionId":"0000000000000000","Message":"RequestPath:/api/v1/health/ready RequestId:0000000000000000","RequestPath":"/api/v1/health/ready","Message":"System.Collections.Generic.Dictionary`2[System.String,System.Object]","CorrelationId":"memoria-smoke-health"}]}

```

Figura 11.1: Evidencia de petición readiness con identificador de correlación y log JSON correlable en CloudWatch.

Las métricas básicas de ECS, ALB y RDS quedan disponibles en CloudWatch como parte de los servicios gestionados. El módulo de observabilidad añade un dashboard `transitops-dev-api` con paneles para ALB, utilización de ECS, RDS y logs recientes. El dashboard no sustituye a una plataforma APM completa, pero ofrece una superficie operativa suficiente para un entorno dev defendible.

11.3 Alarmas y operación

La operación de Sprint 6 deja de depender solo de comprobaciones manuales. Terraform crea alarmas CloudWatch para:

- errores de aplicación detectados por filtro métrico sobre logs JSON;
- respuestas 5xx del target group del ALB;
- tiempo de respuesta del ALB;
- ausencia de targets sanos;
- utilización de CPU de ECS;
- utilización de memoria de ECS;

- CPU de RDS;
- conexiones de base de datos;
- almacenamiento libre en RDS.

Las alarmas usan `treat_missing_data` conservador para reducir ruido cuando dev está destruido o con `desired_count = 0`. Si se configura `alarm_email`, Terraform crea un topic SNS y una suscripción de email; si no se configura, las alarmas existen sin acciones. Durante la validación de Sprint 6 no se creó topic SNS porque no se proporcionó email.

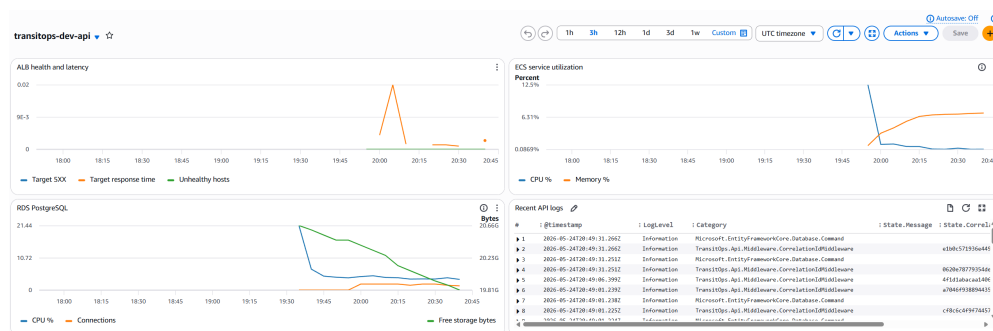


Figura 11.2: Evidencia del dashboard operativo de TransitOps con métricas de ALB, ECS, RDS y logs recientes.

11.4 Seguridad aplicada

Las medidas de seguridad aplicadas son:

- Autenticación mediante login y emisión de **JWT**.
- Contraseñas almacenadas como hash, no en claro.
- Autorización por roles admin y operator.
- Bootstrap del primer administrador protegido por token externo.
- Secretos en Secrets Manager, no en el repositorio.
- RDS en subredes privadas y sin acceso público.
- ECS en subredes privadas, recibiendo tráfico solo desde ALB.
- ALB como único punto de entrada público.
- HTTPS mediante ACM y redirección HTTP a HTTPS.

- GitHub Actions con OIDC en lugar de claves AWS permanentes.
- Correlación de errores y respuestas mediante X-Correlation-ID, evitando exponer trazas internas al cliente.

11.5 Limitaciones de seguridad

El sistema no pretende cubrir todos los controles exigibles a un producto crítico en producción. Quedan fuera del alcance actual la rotación automática de secretos, WAF, protección avanzada contra abuso, auditoría detallada de acciones administrativas, cifrado con claves KMS propias, políticas IAM de mínimo privilegio refinadas al extremo, pruebas de penetración, OpenTelemetry/X-Ray y hardening exhaustivo de cabeceras HTTP.

Estas limitaciones no invalidan el objetivo del trabajo, ya que se ha priorizado una línea base realista y demostrable: autenticación, autorización, red privada, secretos externos, HTTPS e infraestructura reproducible.

11.6 Revisión de seguridad Sprint 8

Sprint 8 añadió una revisión explícita de seguridad sobre la arquitectura desplegable. La conclusión principal es que la aplicación mantiene una frontera pública pequeña: el ALB es el único punto de entrada, HTTP se usa únicamente para redirección a HTTPS, ECS no recibe IP pública y RDS no es accesible desde Internet.

En IAM se diferencian tres casos. El execution role de ECS puede leer únicamente los secretos referenciados por la task definition. El task role de la aplicación no tiene permisos AWS porque la API no llama a servicios AWS en ejecución. El rol OIDC de GitHub conserva permisos amplios sobre los servicios gestionados por Terraform para poder aplicar y destruir el entorno dev; se acepta como riesgo controlado porque la trust policy queda limitada al repositorio y rama configurados, y porque el entorno se destruye tras capturar evidencia.

Check	Passed	Details
-----	-----	-----
AWS account	True	Account=661008947340
RDS private	True	PubliclyAccessible=False
RDS backups disabled for dev	True	BackupRetentionPeriod=0
ECS private networking	True	assignPublicIp=DISABLED
ALB public ports	True	Ports=80,443
ECS ingress from ALB only	True	ECS SG=sg-0d5ac684e66f3c7ff, ALB SG=sg-0be07e059f8e2c454
RDS ingress from ECS only	True	RDS SG=sg-0a3aa05bacd66a906, ECS SG=sg-0d5ac684e66f3c7ff
ECS task role has no inline policies	True	InlinePolicies=
Execution role scoped runtime config	True	Runtime config policy does not use Resource=*
GitHub deploy role broad permissions documented	True	Expected warning: Terraform deploy role uses broad service permissions in dev
Runtime secrets present	True	Secrets=transitops/dev/app/jwt-signing-key,transitops/dev/app/db-connection-string,

Figura 11.3: Evidencia de revisión de seguridad Sprint 8.

Operación y runbooks

12.1 Objetivo operativo

El objetivo operativo de TransitOps es que el entorno dev pueda recrearse, validarse, diagnosticarse y destruirse sin depender de conocimiento tácito. Esta idea se incorporó especialmente en los sprints finales, cuando el proyecto dejó de centrarse únicamente en construir recursos y pasó a demostrar que esos recursos podían operarse con procedimientos claros.

Un runbook no es solo una lista de comandos. En el contexto de este trabajo, cada runbook debe indicar qué problema resuelve, qué pasos principales se ejecutan, qué señales confirman el éxito y qué limpieza debe realizarse. Esta estructura reduce ambigüedad y facilita explicar el proyecto durante la defensa.

La operación se documenta para un entorno efímero. Esto condiciona los procedimientos: muchas acciones terminan con terraform destroy; las alarmas no deben generar ruido cuando dev está apagado; los recursos persistentes intencionales se separan de los recursos del stack de aplicación; y el coste se trata como una variable de diseño, no como una consecuencia posterior.

12.2 Runbook de despliegue normal

El despliegue normal parte de un estado local validado y de un backend Terraform inicializado. El primer paso es aplicar infraestructura con ECS a cero tareas. Esto crea red, balanceador, RDS, ECR, secretos, roles y observabilidad sin arrancar todavía la API. A continuación se construye la imagen Docker, se etiqueta con un identificador operativo y se publica en ECR.

Después de publicar la imagen, se cargan o restauran los secretos de runtime. La API necesita cadena de conexión, clave JWT y token de bootstrap. Sin estos valores, la task definition puede existir, pero la tarea fallará al arrancar. Por ello, la carga de secretos se trata como paso explícito y verificable.

La migración de base de datos se ejecuta como tarea ECS puntual con `--migrate-only`. El criterio de éxito es código de salida 0. Solo después de ese resultado se escala el servicio ECS a `desired_count = 1`. Finalmente se espera estabilidad, se comprueba readiness por HTTPS y se valida login.

Las señales de éxito son: task de migración finalizada, servicio ECS con `desired = 1` y `running = 1`, target group sano, `/api/v1/health/ready` con 200, login administrador con 200, logs JSON en CloudWatch y header `X-Correlation-ID` presente. Si cualquiera de estos pasos falla, el runbook dirige la investigación hacia la capa correspondiente: imagen, secretos, base de datos, ECS, ALB o DNS.

12.3 Runbook de rollback

El rollback manual se basa en volver a un tag de imagen conocido. No se modifica ECS desde consola ni se registra una task definition manual. En su lugar, se actualiza la variable de imagen y se reaplica Terraform. Esta decisión mantiene la infraestructura alineada con el estado versionado y evita que el entorno quede en una configuración difícil de reproducir.

El procedimiento comienza identificando el tag bueno. En Sprint 7, el tag de referencia fue `sprint7-good-3dca035`. El workflow `rollback-dev.yml` acepta el tag, aplica Terraform con `ecs_desired_count = 1`, espera estabilidad de ECS y valida readiness. El resultado esperado es que la task definition activa apunte a la imagen indicada y que el endpoint vuelva a responder 200.

La prueba controlada con un tag inexistente demuestra por qué este runbook es necesario. ECS intentó crear una tarea, la imagen no existía en ECR y la tarea falló con `CannotPullContainerError`. El deployment circuit breaker evitó estabilizar un despliegue incorrecto. La recuperación consistió en reaplicar el tag bueno y comprobar health. Esta evidencia conecta configuración Terraform, comportamiento ECS y operación real.

El cleanup de rollback consiste en dejar el tag bueno como valor deseado y confirmar que no quedan tareas fallidas activas ni cambios pendientes en Terraform. Si el entorno se creó únicamente para la prueba, se ejecuta `destroy` al terminar la validación.

12.4 Runbook de reinicio o redeploy

Un reinicio controlado puede ser necesario si se desea forzar a ECS a reemplazar la tarea actual sin cambiar imagen ni infraestructura. El procedimiento operativo consiste en usar el servicio ECS o el workflow de despliegue para forzar una nueva deployment. La señal de éxito es que ECS cree una nueva tarea, el target group la marque como sana y la anterior se drene sin cortar tráfico.

El diseño del target group ayuda a este proceso. La ruta de readiness evita enviar tráfico a una tarea que aún no puede conectar con PostgreSQL. El `deregistration_delay` corto en dev reduce el tiempo necesario para retirar una tarea antigua durante validaciones. El `stopTimeout` da margen al contenedor para finalizar de forma ordenada.

Si el reinicio falla, las primeras comprobaciones son logs de CloudWatch, eventos del servicio ECS y estado del target group. Un error de secretos suele aparecer como fallo de arranque; un error de base de datos suele reflejarse en readiness; un problema de imagen aparece en eventos de ECS o ECR. El runbook separa estas causas para evitar revisar componentes al azar.

12.5 Runbook de bootstrap de administrador

El bootstrap crea el primer administrador cuando el sistema no tiene ninguno activo. El token de bootstrap se configura fuera del repositorio y se envía en la petición inicial. Si no existe administrador, la respuesta esperada es 201. Si ya existe, la respuesta esperada es 409. Ambos resultados son aceptables según el estado previo de la base de datos.

Este comportamiento es importante para recreaciones. Tras un despliegue desde base nueva, el bootstrap debe permitir crear el primer usuario. Tras una recreación sobre una base con datos ya migrados o restaurados, puede devolver conflicto porque el administrador ya existe. El runbook no trata 409 como fallo automático; lo interpreta como señal de que el sistema ya está inicializado.

Después del bootstrap, el login debe devolver 200 y un JWT. Esta prueba confirma que el usuario existe, que el hash de contraseña se valida correctamente, que la configuración JWT es válida y que la API puede responder peticiones de autenticación en cloud.

12.6 Runbook de restore RDS

El restore de RDS se validó mediante un procedimiento temporal. El objetivo no era conservar una nueva base, sino demostrar que un snapshot manual puede restaurarse y que la aplicación puede ejecutar migraciones contra esa base restaurada. Esta diferencia es relevante: una restauración que solo crea una instancia no prueba que el backend pueda usarla.

El script `Invoke-RdsRestoreTest.ps1` crea un snapshot manual de la base principal, restaura una instancia temporal, crea un secreto temporal con la cadena de conexión, registra una task definition temporal que apunta a ese secreto y ejecuta `--migrate-only`. El criterio de éxito es exit code 0. La prueba detectó que el execution role de ECS necesitaba permiso temporal para leer el nuevo secreto, y el script se corrigió para añadir y retirar esa política durante la prueba.

El cleanup es obligatorio: eliminar la base restaurada, el secreto temporal, la task definition temporal, la política IAM temporal y el snapshot manual salvo que se pida conservar evidencia. Esta limpieza evita coste residual, especialmente porque una instancia RDS restaurada y un snapshot manual pueden seguir generando gasto.

12.7 Runbook de logs por correlation id

La investigación de una petición concreta parte del header X-Correlation-ID. El cliente puede proporcionar un valor conocido o leer el valor devuelto por la API. Ese identificador se usa para filtrar en CloudWatch Logs. El objetivo es encontrar todas las entradas asociadas a una petición sin depender de hora aproximada o de texto del mensaje.

El middleware de correlación alinea el header con `HttpContext.TraceIdentifier` y abre un scope de logging. Como los logs se emiten en JSON con scopes incluidos, CloudWatch puede filtrar campos como `State.CorrelationId`. Esto convierte la correlación en una herramienta práctica durante smoke tests y errores.

El procedimiento de diagnóstico consiste en reproducir la petición con un correlation id controlado, comprobar el código HTTP, abrir CloudWatch Logs Insights, filtrar por ese identificador y revisar método, ruta, estado, duración y nivel de log. Si el error es interno, el cliente recibe una respuesta controlada con request id y el servidor conserva el detalle en logs.

12.8 Runbook de alarmas

Las alarmas creadas por Terraform cubren señales básicas de ALB, ECS, RDS y errores de aplicación. Cuando una alarma se dispara, el primer paso es identificar la capa afectada. Una alarma de 5xx del ALB puede indicar errores de aplicación o problemas de target. Una alarma de response time puede apuntar a lentitud en API o base de datos. Una alarma de hosts no sanos apunta a ECS, target group o readiness. Las alarmas de CPU/memoria se relacionan con capacidad de contenedor o base de datos.

El runbook recomienda revisar primero el dashboard CloudWatch para obtener contexto general. Después se consultan logs por rango temporal y, si existe una petición concreta, por correlation id. Para problemas de ECS se revisan eventos del servicio y estado de tasks. Para problemas de RDS se revisan conexiones, CPU y almacenamiento libre.

En dev, las alarmas se configuran con tratamiento conservador de datos ausentes. Esto evita alertas falsas cuando el entorno está destruido o con ECS a cero tareas. Si se configura `alarm_email`, Terraform crea topic SNS y suscripción; la suscripción puede quedar en `PendingConfirmation` hasta aceptar el correo. Este estado se documenta como esperado.

12.9 Runbook de destroy y auditoría

El cierre de coste es una operación obligatoria tras validaciones cloud. El procedimiento principal es ejecutar `terraform destroy` desde la raíz del entorno dev. Antes del destroy puede ejecutarse `terraform plan -detailed-exitcode` para confirmar si existen cambios pendientes. Después del destroy, el state debe quedar vacío.

La auditoría post-destroy comprueba que no quedan recursos efímeros del stack: ALB, target groups, ECS, ECR, RDS, NAT Gateway, certificados ACM, registros Route 53 del subdominio, log groups, dashboards, alarmas, secretos runtime y snapshots temporales. Sprint 8 añadió un script específico para esta verificación.

No todos los recursos desaparecen. Permanecen intencionadamente el dominio registrado, la hosted zone pública y el backend remoto Terraform en S3/DynamoDB. Estos recursos son base para poder recrear el entorno y se documentan como persistentes. La separación entre persistente y efímero evita confundir un destroy correcto con la eliminación total de la cuenta AWS.

12.10 Aprendizajes operativos

Los sprints finales mostraron que muchas dificultades reales no aparecen en una arquitectura dibujada. ACM puede quedar en validación pendiente si el dominio no delega en la hosted zone correcta. Secrets Manager puede bloquear la recreación inmediata de un secreto si quedó programado para eliminación. ECR puede impedir un destroy si el repositorio contiene imágenes, salvo que se configure `force delete`. Una tarea ECS puede fallar por permisos de lectura de un secreto temporal aunque la infraestructura principal funcione.

Estos casos son valiosos porque convierten la memoria en una descripción de comportamiento real, no solo en una intención de diseño. Documentar incidencias y correcciones aporta más valor que presentar un flujo idealizado. El proyecto demuestra que operar cloud implica entender estados intermedios, permisos, propagación DNS, recursos residuales y coste.

Por ello, los runbooks forman parte del resultado del TFG. No son documentación secundaria, sino una forma de cerrar el ciclo entre código, infraestructura y operación. En una defensa técnica, permiten explicar no solo qué se construyó, sino cómo se recupera, cómo se diagnostica y cómo se evita dejar gasto activo.

Validación y resultados

13.1 Estrategia de validación

La validación se ha organizado en tres niveles: pruebas de backend, validación de infraestructura y pruebas de operación sobre el entorno desplegado. Esta estrategia evita depender de una sola evidencia y permite demostrar tanto comportamiento funcional como capacidad real de despliegue.

En local se ejecutan pruebas automatizadas y flujos Postman/Newman contra una API viva. En infraestructura se usan `terraform fmt`, `terraform validate`, `terraform plan`, `terraform apply` y `terraform destroy`. En AWS se comprueba el estado de ECS, la migración puntual, la salud del endpoint, el bootstrap de administrador, el login, la existencia de recursos de observabilidad, el rollback de despliegue y el restore temporal de RDS.

13.2 Pruebas funcionales

La suite principal de pruebas automatizadas contiene 87 pruebas superadas durante la fase de cierre. Estas pruebas cubren endpoints de salud, transportes, vehículos, conductores, autenticación, usuarios, reglas de estado y el contrato de correlación de peticiones. La validación automatizada se complementa con un flujo de smoke local basado en Postman/Newman, que arranca Docker Compose, prepara datos deterministas, ejecuta peticiones y limpia los datos generados.

Además, se verificaron manualmente el bootstrap del primer administrador y el login en el entorno cloud.

13.3 Validación de infraestructura

La infraestructura se validó con Terraform antes y después del despliegue. Las acciones principales fueron:

- Inicialización de backend remoto S3/DynamoDB.
- terraform validate correcto para el entorno dev.
- terraform apply real de la infraestructura dev.
- Publicación de imagen en ECR.
- Aplicación de migraciones EF Core mediante ECS task.
- Escalado del servicio ECS a una tarea.
- Activación del camino DNS/HTTPS con Route 53 y ACM.
- Creación de dashboard, alarmas y filtro métrico de errores.
- Prueba de rollback mediante tag de imagen conocido.
- Prueba de fallo controlado con tag inexistente y recuperación al tag bueno.
- Prueba de restore RDS desde snapshot manual y migración contra base restaurada.
- terraform plan con -detailed-exitcode sin cambios pendientes antes del cierre de coste.
- terraform destroy para eliminar recursos costosos del entorno dev.

13.4 Resultados obtenidos

Los resultados técnicos obtenidos en la validación cloud son:

- Endpoint de validación Sprint 6: DNS público del ALB registrado como evidencia operativa.
- Endpoint objetivo DNS/HTTPS: <https://api.dev.transitops.net>, gestionado por Route 53 y ACM cuando el entorno se recrea con HTTPS activo.
- Health check Sprint 6: GET /api/v1/health/ready devuelve 200.
- Certificado ACM: validación DNS corregida en la cuenta 661000947340; el certificado para api.dev.transitops.net llegó a estado ISSUED antes del destroy.

- Servicio ECS: ACTIVE, desired = 1, running = 1.
- Imagen ECR desplegada en Sprint 6: tag operativo sprint6-local-6bb910c.
- Task definition desplegada: revisión transitops-dev-api:3.
- Tarea ECS de migración finalizada con código 0.
- Bootstrap de administrador: respuesta 201.
- Login administrador: respuesta 200 con JWT.
- Header de correlación: X-Correlation-ID presente en health, errores y login.
- CloudWatch Logs: entradas JSON filtrables por State.CorrelationId, incluyendo la petición sprint6-login-final.
- Observabilidad: dashboard CloudWatch, log group de la API y 9 alarmas creadas por Terraform.
- SNS: en Sprint 6 no se creó topic al no estar configurado alarm_email; en las recreaciones posteriores se configuró pablomlopez03@gmail.com y la suscripción quedó en PendingConfirmation, estado esperado hasta confirmar el correo.
- Cierre de coste: terraform destroy eliminó los recursos del entorno dev; el state quedó vacío.

Estos resultados demuestran que el sistema no solo compila, sino que se ejecuta sobre infraestructura cloud real y verificable, y que también puede destruirse de forma controlada para evitar gasto residual.

13.5 Validación Sprint 7

Sprint 7 añadió una validación real de fiabilidad sobre la cuenta AWS 661000947340, en la región eu-west-1. La imagen buena publicada en ECR fue sprint7-good-3dca035. La tarea ECS de migración terminó con exit code 0, el servicio quedó en desired = 1 y running = 1, y la task definition activa fue transitops-dev-api:4. Los identificadores completos de tarea y digest quedan recogidos en la documentación operativa para no sobrecargar el cuerpo de la memoria.

Las comprobaciones principales fueron:

- Readiness HTTPS: GET sobre la ruta de readiness con respuesta 200 y correlation id sprint7-health-good.

- Bootstrap de administrador: respuesta 201 con correlation id sprint7-bootstrap-good.
- Login administrador: respuesta 200 con correlation id sprint7-login-good.
- Observabilidad: log group de la API, dashboard transitops-dev-api, 9 alarmas Cloud-Watch y logs JSON filtrables por State.CorrelationId.
- SNS: topic y suscripción para pablomlopez03@gmail.com; la suscripción quedó en PendingConfirmation, estado esperado hasta confirmar el correo.
- Validación Docker local: build correcto y usuario no root en runtime, verificado con id.

La prueba de rollback usó el tag inexistente sprint7-missing-3dca035. ECS creó una tarea que falló con CannotPullContainerError al no existir el manifiesto de imagen en ECR. Después se reaplicó Terraform con sprint7-good-3dca035 y el endpoint de readiness volvió a responder 200, con correlation id sprint7-health-after-bad-image.

La prueba de restore de RDS creó un snapshot manual, restauró una instancia temporal, creó un secreto temporal de conexión y ejecutó una tarea ECS --migrate-only contra la base restaurada. La task definition temporal transitops-dev-api-restore-sprint7:2 terminó con exit code 0. Durante esta validación se detectó una mejora real: el secreto temporal también necesita permiso secretsmanager:GetSecretValue en el execution role de ECS. El script fue corregido para añadir y eliminar una política IAM temporal como parte de la prueba.

13.6 Evidencia de cierre de coste

Tras la validación de Sprint 6 se ejecutó terraform destroy. El primer intento eliminó la mayoría de recursos y falló al borrar ECR porque el repositorio contenía imágenes. Se vació el repositorio con aws ecr batch-delete-image y se repitió el destroy, que finalizó correctamente.

Tras Sprint 7 se repitió el cierre con la mejora ecr_force_delete = true. El destroy final eliminó 72 recursos y terraform state list no devolvió ningún recurso. Después se verificó que:

- terraform state list no devolvía recursos.
- No quedaban ALB, RDS, ECS ni ECR asociados a transitops-dev.
- El NAT Gateway aparecía en estado deleted.
- No quedaban certificados ACM ni registros Route 53 de api.dev.transitops.net.
- No quedaban log groups, dashboard, alarmas, snapshots temporales ni secretos del entorno dev.

```
PS C:\Users\pey\Projects\TransitOps> aws ecs describe-services `
>> --cluster $cluster `
>> --services $service `
>> --region $region `
>> --profile $profile `
>> --query "services[0].[serviceName,status,desiredCount,runningCount]" `
>> --output table
-----+-----
| DescribeServices |
+-----+-----+
| transitops-dev-api-svc |
| ACTIVE              |
| 1                   |
| 1                   |
+-----+-----+
```

```
PS C:\Users\pey\Projects\TransitOps> curl.exe -i `
>> -H "X-Correlation-ID: $cid" `
>> https://api.dev.transitops.net/api/v1/health/ready
HTTP/1.1 200 OK
Date: Sun, 24 May 2026 21:01:26 GMT
Content-Type: application/json; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
Server: Kestrel
X-Correlation-ID: memoria-logs-20260524224644

{"data":{"status":"ready","service":"TransitOps.Api","environment":"Production",
,"pagination":null}}
```

Figura 13.1: Evidencia de servicio cloud operativo: ECS estable y readiness HTTPS con respuesta 200.

Permanecen fuera de este destroy el dominio registrado transitops.net, su hosted zone pública y el backend remoto de Terraform, ya que son recursos de base necesarios para recreaciones posteriores y no forman parte del stack efímero de aplicación.

13.7 Validación Sprint 8

Sprint 8 completa la validación operativa con dos scripts de auditoría y un conjunto de runbooks. La recreación real partió de un state vacío, creó 72 recursos con ECS a cero tareas, publicó la imagen sprint8-good-fdf98b9, cargó secretos runtime y ejecutó la tarea ECS de migración con exit code 0. Después se escaló el servicio a desired = 1 y running = 1, usando la task definition transitops-dev-api:9.

La validación funcional confirmó readiness HTTPS 200 con correlation id sprint8-health, bootstrap de administrador 201 con sprint8-bootstrap y login 200 con sprint8-login. CloudWatch contenía logs JSON filtrables por ese identificador, el dashboard transitops-dev-api, 9 alarmas, topic SNS y suscripción de email en estado PendingConfirmation. Antes de destruir, terraform plan -detailed-exitcode no mostró cambios pendientes.

La auditoría de seguridad verificó que RDS no era público, ECS no tenía IP pública, los grupos de seguridad mantenían el flujo Internet-ALB-ECS-RDS, los secretos runtime existían en Secrets Manager y el rol de aplicación no contenía políticas inline. El rol OIDC de despliegue conserva permisos amplios para ejecutar Terraform en dev; se documenta como riesgo aceptado porque la trust policy queda limitada al repositorio y rama configurados.

También se revisa el coste del entorno. Los recursos que generan coste mientras dev está activo son principalmente NAT Gateway, ALB, ECS Fargate, RDS, CloudWatch, Secrets Manager y ECR. El cierre obligatorio del sprint volvió a ejecutar terraform destroy, eliminó 72 recursos y después comprobó la ausencia de ALB, ECS, RDS, ECR, certificado ACM, registros api.dev, logs, dashboard, alarmas, secretos y snapshots temporales. Los secretos runtime se borraron de forma forzada tras quedar inicialmente programados para eliminación por Secrets Manager.

13.8 Análisis de limitaciones

Las principales limitaciones son:

- No existe interfaz gráfica; el sistema se consume mediante API, Postman o clientes equivalentes.
- El entorno desplegado es dev, no una arquitectura productiva multi-entorno completa.
- No se han implementado políticas de autoscaling ni pruebas de carga.

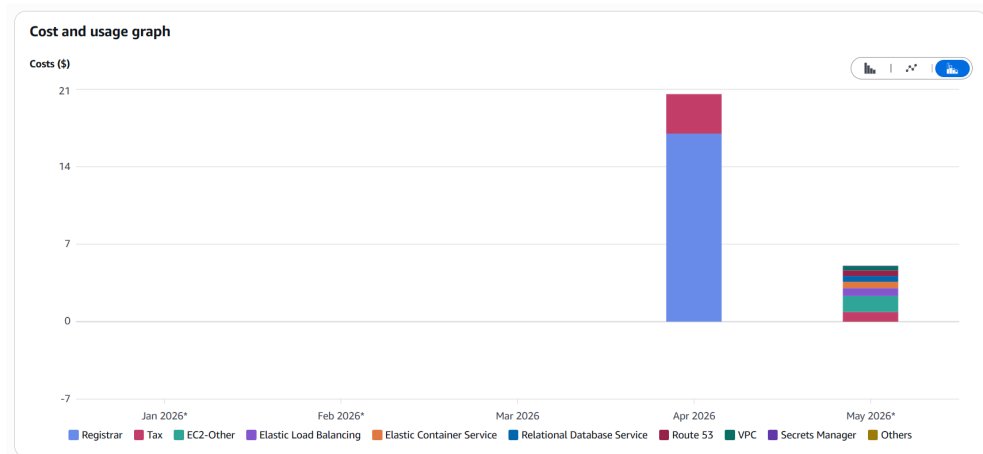


Figura 13.2: Evidencia de revisión de coste Sprint 8.

Check	Passed	Details
No ECS dev cluster	True	
No RDS dev database	True	
No temporary restore database	True	
No ECR dev repository	True	
No API log group	True	None
No CloudWatch dashboard	True	None
No CloudWatch alarms	True	
No ACM API certificate	True	None
No runtime secrets	True	
No manual dev snapshots	True	
No dev ALB	True	
No api.dev Route53 records	True	

Figura 13.3: Evidencia de auditoría post-destroy Sprint 8.

- La observabilidad cubre logs JSON, correlación, dashboard y alarmas básicas, pero no incluye trazas distribuidas ni APM completo.
- La evidencia visual incorporada cubre despliegue, DNS/HTTPS, observabilidad, roll-back, restore, coste y auditoría post-destroy.
- Algunas políticas IAM son prácticas para el sprint actual y pueden afinarse más en fases posteriores.

13.9 Cierre Sprint 9

El cierre final del proyecto organiza la evidencia y la trazabilidad de forma explícita. Se añade una matriz que relaciona requisitos funcionales y no funcionales con implementación, pruebas y evidencia documental. También se prepara un índice final de evidencias y un guion de verificación para poder repetir localmente las comprobaciones principales o, si se necesitan capturas frescas, recrear brevemente el entorno dev, validar HTTPS, login, observabilidad y destruirlo de nuevo.

Los documentos principales de cierre son:

- docs/RequirementsTraceability.md: matriz FR/NFR frente a implementación, verificación y evidencia.
- docs/FinalEvidence.md: índice de evidencias reales y capturas incorporadas.
- docs/FinalVerification.md: checklist local, ruta opcional de recreación AWS y guion de defensa técnica.

Este cierre evita que la defensa dependa de memoria de sesión o de una explicación oral improvisada. La relación entre alcance, implementación, operación y evidencias queda registrada en el repositorio.

Conclusiones

14.1 Conclusiones del trabajo

El trabajo ha alcanzado un estado técnico avanzado respecto a sus objetivos principales. Se ha construido un backend funcional para gestión de transportes y se ha desplegado en AWS mediante infraestructura como código, con base de datos privada, contenedores, secretos externos, configuración DNS/HTTPS, observabilidad en CloudWatch y validación operativa.

El valor principal del proyecto no reside únicamente en la API implementada, sino en haber completado un camino de ingeniería real: código fuente, pruebas, imagen Docker, ECR, Terraform, RDS, ECS, migraciones, DNS, TLS, logs estructurados, correlación de peticiones, dashboard, alarmas, health check, bootstrap, login validado y destrucción controlada del entorno para evitar coste innecesario. Esto permite defender el trabajo como una plataforma cloud operable, no solo como una aplicación local.

14.2 Competencias aplicadas

Durante el desarrollo se han aplicado competencias de:

- Ingeniería de requisitos y control de alcance.
- Diseño de API REST y contratos de error.
- Modelado de dominio y reglas de negocio.
- Persistencia relacional con PostgreSQL y migraciones.
- Pruebas automatizadas e integración.
- Contenerización con Docker.
- Infraestructura como código con Terraform.

- Diseño de red y servicios cloud en AWS.
- CI/CD con GitHub Actions y OIDC.
- Seguridad básica de aplicación e infraestructura.
- Observabilidad con logs estructurados, correlación, métricas y alarmas.
- Operación y control de costes mediante ciclos de creación, validación y destrucción de infraestructura.
- Documentación técnica y preparación de evidencias.
- Trazabilidad final entre requisitos, implementación, pruebas y evidencias.

14.3 Lecciones aprendidas

Una lección importante es que la complejidad real de un sistema cloud no aparece solo en el código de negocio. DNS, certificados, roles, secretos, migraciones, estado Terraform, observabilidad, limpieza de recursos y permisos pueden bloquear un despliegue si no se tratan como parte del diseño.

También se ha comprobado el valor de mantener un alcance funcional reducido. Esta decisión permitió dedicar tiempo a cerrar aspectos que a menudo quedan fuera de proyectos académicos: despliegue real, HTTPS, base de datos privada, migraciones controladas, observabilidad, alarmas, rollback de despliegue y validación de extremo a extremo.

Por último, el proyecto muestra que documentar decisiones durante el desarrollo facilita retomar el trabajo, justificar cambios y preparar la defensa.

14.4 Líneas futuras

Las líneas futuras más relevantes son:

- Desarrollar un frontend web para operadores y administradores.
- Añadir entorno prod separado, con configuración y estado Terraform independientes.
- Refinar permisos IAM hacia mínimo privilegio estricto.
- Evolucionar la observabilidad hacia trazas distribuidas, métricas de negocio y runbooks asociados a cada alarma.
- Incorporar pruebas de carga y análisis de capacidad.

- Implementar rotación de secretos y políticas de backup/restore verificadas.
- Ampliar el dominio con planificación, costes, rutas o integración con servicios externos.

Apéndices

Material complementario

A.1 Anteproyecto

El anteproyecto firmado se conserva en el repositorio de la memoria como referencia documental del alcance aprobado.

A.2 Evidencias técnicas

Las siguientes evidencias quedan incorporadas en la memoria o documentadas como soporte operativo del repositorio:

Cuadro A.1: Capturas y evidencias incorporadas

Archivo o evidencia	Contenido
imaxes/route53-dns-evidence.png	Hosted zone transitops.net, nameservers delegados y registro api.dev.transitops.net.
imaxes/acm-issued.png	Certificado ACM de api.dev.transitops.net en estado ISSUED.
imaxes/deploy-cloud-evidence.png	Imagen en ECR, servicio ECS estable y readiness HTTPS 200.
imaxes/rollback-tag-evidence.png	Task definition activa apuntando al tag operativo validado.
imaxes/recreate-terraform-outputs.png	Outputs de Terraform tras recrear dev.

imagenes/rollback-restore-evidence.png	Rollback automático y restore RDS con migración terminada correctamente.
imagenes/cloudwatch-logs-correlation.png	Petición readiness y log JSON correlado en CloudWatch.
imagenes/cloudwatch-dashboard.png	Dashboard CloudWatch transitops-dev-api.
imagenes/security-review.png	Ejecución de Test-Sprint8AwsPosture.ps1.
imagenes/cloud-smoke-evidence.png	ECS 1/1 y readiness HTTPS 200.
imagenes/cost-review.png	Vista de coste por servicio durante una recreación temporal del entorno dev.
imagenes/destroy-audit.png	Auditoría posterior a terraform destroy con ausencia de recursos efímeros.

El índice operativo de evidencias queda además documentado en docs/FinalEvidence.md, junto con las rutas y comandos que permiten reproducir las comprobaciones principales.

Trazabilidad de requisitos y evidencias

B.1 Propósito del anexo

Este anexo resume la relación entre requisitos, implementación, validación y evidencia. Su objetivo es facilitar la defensa del proyecto y evitar que la evaluación dependa únicamente de una explicación oral. Cada requisito importante puede relacionarse con una parte del código, un mecanismo de prueba y una evidencia documental o cloud.

La trazabilidad no sustituye a los documentos completos del repositorio. La matriz operativa principal se mantiene en docs/RequirementsTraceability.md; este anexo recoge una versión integrada en la memoria para que el PDF sea autocontenido.

B.2 Requisitos funcionales

Cuadro B.1: Trazabilidad de requisitos funcionales

ID	Implementación	Validación	Evidencia
RF-01	Endpoints de salud /api/v1/health/live /api/v1/health/ready.	Tests de health, smoke local y smoke cloud.	Health HTTPS 200, target group sano y logs de readiness.
RF-02	Endpoint de bootstrap protegido por token externo.	Pruebas de creación inicial y conflicto si ya existe administrador.	Bootstrap cloud 201 o 409 documentado.
RF-03	Login con usuario y contraseña, emisión de JWT.	Tests de autenticación y smoke cloud de login.	Login administrador 200 con JWT.

RF-04	Autorización por roles admin y operator.	Tests de endpoints protegidos y acceso denegado.	Casos de prueba y documentación de roles.
RF-05	Endpoints de administración de usuarios.	Tests de creación, listado, cambio de rol y activación.	Suite automatizada de backend.
RF-06	Gestión de transportes con filtros, paginación y borrado lógico.	Tests de CRUD, validación y soft delete.	Pruebas de integración.
RF-07	Gestión de vehículos asignables.	Tests de validación de matrícula, unicidad y borrado lógico.	Pruebas de dominio y persistencia.
RF-08	Gestión de conductores asignables.	Tests de licencia, contacto, estado activo y unicidad.	Pruebas de integración.
RF-09	Asignación de vehículo y conductor a transporte.	Tests de asignación válida e inválida.	Validación de reglas de negocio.
RF-10	Transiciones de estado controladas.	Tests de estados terminales y precondiciones.	Suite de reglas de transporte.
RF-11	Registro y consulta de eventos logísticos.	Tests de creación y consulta cronológica.	Historial de eventos validado.
RF-12	Errores coherentes y contrato común.	Tests de validación, no encontrado, conflicto y autorización.	Respuestas HTTP y request id correlable.

B.3 Requisitos no funcionales

Cuadro B.2: Trazabilidad de requisitos no funcionales

Área	Implementación	Validación	Evidencia
Reprod. local	Docker Compose con API y PostgreSQL.	Smoke local y pruebas automatizadas.	Comandos documentados en README y validación local.
Mantenibilidad	Solución simple con proyectos TransitOps.Api y TransitOps.Tests.	Revisión de estructura y tests.	Capítulos de arquitectura e implementación.
Persistencia	PostgreSQL y migraciones EF Core.	Migraciones locales y ECS --migrate-only.	Tareas ECS de migración con exit code 0.
Seguridad básica	Hash de contraseñas, JWT, roles, secretos externos y red privada.	Tests de auth, auditoría Sprint 8 y revisión de SG/IAM.	Script de posture audit y documentación de seguridad.

IaC	Terraform con módulos y estado remoto.	terraform fmt, validate, apply y destroy.	Recreate-from-scratch y state vacío tras destroy.
Observab.	Logs JSON, correlation id, dashboard, alarmas y filtro métrico.	Smoke con correlation id y verificación CloudWatch.	Dashboard, 9 alarmas y logs filtrables.
CI/CD	Workflows manuales de Terraform, deploy y rollback.	Ejecución local, publicación de imagen y validación cloud.	Documentación de pipelines y capturas de ECR, ECS, readiness y task definition activa.
Operación	Runbooks de deploy, rollback, restore, logs, alarmas y destroy.	Sprint 7 y Sprint 8 con AWS real.	docs/CloudReliability.md, docs/CloudDeployment.md.
Coste	Entorno dev efímero, backups RDS a cero y destroy final.	Auditoría post-destroy.	Ausencia de recursos efímeros costosos.

B.4 Evidencias por sprint

Cuadro B.3: Resumen de evidencias por sprint

Sprint	Objetivo	Evidencia generada
Sprint 1	MVP local backend.	Endpoints funcionales, modelo de dominio, pruebas iniciales y ejecución local.
Sprint 2	Base Terraform y red AWS.	Estructura de módulos, remote state, VPC, subredes, NAT y grupos de seguridad.
Sprint 3	Runtime cloud planificado.	ECR, RDS, ECS, ALB, Route 53, ACM, Secrets Manager y CloudWatch definidos.
Sprint 4	Primer despliegue real.	Imagen en ECR, migraciones ECS, servicio estable, HTTPS y login admin.
Sprint 5	CI/CD con OIDC.	Workflows de validación y despliegue manual sin claves AWS persistentes.
Sprint 6	Observabilidad y hardening.	Logging JSON, correlation id, dashboard, alarmas, circuit breaker y DNS/HTTPS corregido.
Sprint 7	Rollback, restore y tuning.	Rollback a tag bueno, fallo con tag inexistente, restore RDS temporal y Docker hardening.
Sprint 8	Seguridad, coste y runbooks.	Auditoría de postura, recreate-from-scratch, coste documentado y destroy audit.

Sprint 9

Cierre documental.

Trazabilidad, índice final de evidencias, guion de demo y memoria final.

B.5 Relación entre evidencia y defensa

Las evidencias se agrupan en tres niveles. El primer nivel corresponde a evidencias locales: tests automatizados, build Docker, usuario no root y validación de Terraform. Estas pruebas demuestran que el repositorio es coherente y que el sistema puede construirse sin depender de AWS.

El segundo nivel corresponde a evidencias cloud: Terraform apply, imagen ECR, migración ECS, servicio estable, health HTTPS, login, logs correlados, dashboard, alarmas, rollback y restore. Estas evidencias demuestran que el proyecto no se queda en diseño teórico y que se ejecutó sobre servicios reales.

El tercer nivel corresponde a evidencias de cierre: destroy final, state vacío, ausencia de recursos costosos, coste revisado y recursos persistentes justificados. Este nivel es importante porque una arquitectura cloud académica no debe ignorar el gasto ni dejar recursos activos por descuido.

Durante la defensa, estas evidencias permiten recorrer el proyecto desde requisitos hasta operación. Un posible guion consiste en explicar primero el problema y el alcance, después enseñar la arquitectura y el flujo de despliegue, luego validar una petición con correlation id y finalmente mostrar rollback, restore y destroy como pruebas de madurez operativa.

B.6 Capturas incorporadas

La memoria incorpora capturas reales de las validaciones cloud y de las auditorías finales. Las evidencias principales incluyen:

- Servicio ECS estable con una tarea en ejecución.
- Certificado ACM en estado Issued.
- Route 53 con el registro del subdominio y validación ACM.
- Dashboard CloudWatch.
- Consulta de logs por X-Correlation-ID.
- Publicación en ECR, task definition activa y readiness HTTPS.
- Prueba de restore RDS.

- Auditoría de seguridad Sprint 8.
- Cost Explorer durante recreación temporal.
- Auditoría post-destroy.

Estas capturas permiten defender el recorrido completo: despliegue, operación, observabilidad, recuperación, coste y cierre del entorno.

Procedimientos operativos

C.1 Recreación completa de dev

La recreación completa del entorno dev parte de un estado destruido. El objetivo es demostrar que el proyecto no depende de recursos creados manualmente ni de una configuración conservada accidentalmente. El procedimiento comienza validando la cuenta AWS, la región y el perfil local. A continuación se inicializa Terraform con backend remoto y se aplica la infraestructura con ECS a cero tareas.

El siguiente paso es construir y publicar una imagen Docker. El tag debe ser identificable, por ejemplo `sprint9-final-<sha>` en una validación final. Después se cargan secretos runtime en Secrets Manager y se ejecuta una tarea ECS de migración mediante `--migrate-only`. Si la migración termina con exit code 0, se escala el servicio a una tarea y se espera estabilidad.

La validación final incluye readiness HTTPS, bootstrap o confirmación de administrador existente, login, logs con correlation id, dashboard, alarmas y postura de seguridad mínima. Si el objetivo era obtener evidencia, se capturan las pantallas necesarias y se ejecuta destroy al terminar.

C.2 Comprobaciones locales previas

Antes de recrear AWS, el repositorio debe validarse localmente. Las comprobaciones recomendadas son:

- Ejecutar `dotnet test` sobre el proyecto de pruebas.
- Ejecutar `terraform fmt -check -recursive` en la carpeta de infraestructura.
- Ejecutar `terraform validate` en el entorno dev.
- Construir la imagen Docker de la API.

- Verificar que el contenedor no se ejecuta como root.
- Buscar patrones básicos de secretos en el repositorio.
- Compilar la memoria con latexmk.

Estas comprobaciones reducen el riesgo de descubrir errores triviales durante el apply cloud, donde cada minuto puede generar coste. Además, separan fallos de código, fallos de infraestructura y fallos de AWS.

C.3 Carga de secretos runtime

Los secretos runtime no se versionan. Para que ECS arranque correctamente, Secrets Manager debe contener al menos la cadena de conexión de PostgreSQL, la clave de firma JWT y el token de bootstrap. Terraform conoce los nombres y permisos necesarios, pero no debe incluir los valores reales.

Cuando un secreto queda programado para eliminación, AWS puede impedir recrearlo con el mismo nombre hasta que expire el periodo de recuperación. Durante los sprints se resolvió esta situación restaurando o eliminando de forma forzada los secretos según el caso. Esta incidencia queda documentada porque afecta directamente a ciclos de destroy y recreate.

La señal de éxito de esta fase no es solo que el secreto exista, sino que la task definition pueda leerlo. Si ECS falla al arrancar con errores de acceso a secretos, se revisan permisos del execution role y ARN referenciados.

C.4 Migraciones ECS

La migración cloud se ejecuta como tarea ECS puntual. Esta tarea usa la misma imagen de la API, pero cambia el modo de ejecución a --migrate-only. Así se reutiliza el artefacto desplegable y se evita mantener una imagen separada para migraciones.

El criterio de éxito es que la tarea termine con código 0. Si falla, no se escala el servicio web. Esta regla evita poner tráfico sobre una aplicación cuyo esquema de base de datos no está preparado. Los logs de la tarea se revisan en CloudWatch para diagnosticar problemas de conexión, permisos o migraciones.

La misma estrategia se usa en la prueba de restore. En ese caso, la task definition temporal apunta a un secreto de conexión de la base restaurada. Si la migración termina correctamente, se demuestra que la base restaurada es utilizable por la aplicación.

C.5 Validación HTTPS y DNS

El endpoint objetivo es `https://api.dev.transitops.net`. Para que funcione, deben alinearse Route 53, ACM y ALB. La hosted zone debe estar delegada desde el dominio registrado; ACM debe poder validar el CNAME; el certificado debe estar en estado ISSUED; y el listener HTTPS del ALB debe usar ese certificado.

La incidencia de Sprint 6 mostró que crear una hosted zone no basta. El dominio registrado tenía nameservers distintos de los de la hosted zone usada por Terraform, por lo que ACM no podía validar públicamente el CNAME. Al corregir la delegación, el certificado pasó a ISSUED. Esta experiencia se incorporó a la documentación de despliegue.

La validación DNS/HTTPS se confirma con readiness HTTPS. Si HTTPS falla, se revisan por orden: resolución DNS, estado del certificado, listeners del ALB, target group y health de ECS.

C.6 Auditoría de seguridad

La auditoría mínima de seguridad revisa que RDS no sea público, ECS no tenga IP pública, los security groups sigan el flujo esperado, los secretos estén en Secrets Manager y el rol de aplicación no tenga permisos AWS innecesarios. También se revisa el rol OIDC de GitHub, aceptando que mantiene permisos amplios para aplicar y destruir dev, pero con trust policy limitada.

El resultado esperado no es una certificación de seguridad completa. El objetivo es demostrar que el entorno no expone accidentalmente base de datos ni contenedores, que el único punto público es el ALB y que los secretos no están en el repositorio. Los riesgos aceptados se documentan: no hay WAF, no hay rotación automática de secretos y no se han refinado todos los permisos al mínimo absoluto.

C.7 Auditoría de coste

La auditoría de coste identifica recursos que generan gasto durante una recreación: NAT Gateway, ALB, ECS Fargate, RDS, CloudWatch, Secrets Manager, ECR y Route 53. El proyecto acepta este coste de forma temporal durante validaciones, pero exige destruir el entorno al terminar.

Después del destroy se comprueba ausencia de recursos efímeros. ECR se configura con force delete en dev para que imágenes de validación no bloqueen la destrucción. RDS usa skip_final_snapshot y backups automáticos en cero días para evitar coste residual. Los snapshots manuales de pruebas de restore se eliminan salvo que se conserve evidencia explícitamente.

Permanecen intencionadamente el dominio, la hosted zone y el backend remoto. Estos recursos son necesarios para recrear el entorno y su coste es bajo en comparación con NAT, ALB o RDS.

C.8 Guion de defensa técnica

Un guion razonable de defensa puede seguir este orden:

1. Presentar problema, alcance y decisión de mantener funcionalidad pequeña.
2. Explicar el modelo de dominio y los casos de uso principales.
3. Mostrar la arquitectura cloud: ALB público, ECS privado y RDS privado.
4. Explicar Terraform, módulos y estado remoto.
5. Recorrer el flujo de despliegue: imagen, ECR, migración, ECS y health.
6. Mostrar observabilidad: correlation id, logs JSON, dashboard y alarmas.
7. Explicar rollback con tag bueno y fallo controlado.
8. Explicar restore RDS con snapshot temporal y migración.
9. Presentar seguridad y coste: secretos, red privada, HTTPS y destroy.
10. Cerrar con limitaciones y líneas futuras.

Este guion permite defender el proyecto como una práctica completa de ingeniería cloud. La API no pretende ser un producto logístico completo, pero sí demuestra el ciclo de vida técnico de un backend moderno.

C.9 Checklist de cierre

La checklist final resume las condiciones que deben cumplirse antes de considerar cerrado el proyecto:

- Tests de backend superados.
- Terraform formateado y validado.
- Imagen Docker construida correctamente.
- Usuario no root verificado en contenedor.

- Memoria compilada sin errores.
- Requisitos y trazabilidad actualizados.
- Evidencias visuales y documentales registradas.
- Si se recrea AWS, health HTTPS y login validados.
- Observabilidad comprobada.
- Destroy ejecutado tras validación cloud.
- Auditoría post-destroy sin recursos efímeros costosos.

Esta checklist conecta la parte académica y la parte operativa. No basta con que el código exista; debe poder probarse, explicarse, desplegarse y cerrarse sin dejar coste evitable.

C.10 Diagnóstico por capas

Cuando una validación falla, el enfoque más útil es diagnosticar por capas. La primera capa es el código de aplicación. Si los tests automatizados fallan, no tiene sentido continuar hacia Docker o AWS. Se revisan controladores, servicios de aplicación, reglas de dominio, configuración de tests y migraciones. Esta capa permite aislar errores funcionales antes de que se mezclen con problemas de infraestructura.

La segunda capa es el empaquetado. Si el código compila pero la imagen Docker no arranca, se revisa el Dockerfile, el usuario de ejecución, el puerto expuesto, las variables de entorno y las dependencias de runtime. En TransitOps, la eliminación del puerto innecesario y la ejecución como usuario no root reducen ambigüedad sobre cómo debe comportarse el contenedor.

La tercera capa es la infraestructura base. Aquí se revisan Terraform, VPC, subredes, security groups, ECR, RDS, ALB y secretos. Un error de Terraform puede aparecer antes de crear recursos; un error de permisos puede aparecer al arrancar una task; un error de red puede verse como fallo de readiness; y un error de DNS puede impedir que ACM valide el certificado. Separar estas causas evita interpretar todo fallo cloud como un problema de código.

La cuarta capa es la ejecución ECS. En esta capa se consultan eventos del servicio, estado de tasks, task definition activa, imagen referenciada, secretos inyectados y logs de arranque. Si aparece CannotPullContainerError, la causa probable está en ECR o en el tag de imagen. Si la tarea arranca y luego falla readiness, la causa puede estar en base de datos, configuración o health check. Si la tarea queda sana pero HTTPS falla, el foco se desplaza a ALB, DNS o certificado.

La quinta capa es observabilidad. Una vez que el sistema responde, los logs JSON y el correlation id permiten investigar peticiones concretas. El dashboard aporta contexto de ALB,

ECS y RDS. Las alarmas no sustituyen al análisis, pero señalan qué métrica se salió de los umbrales esperados. Este enfoque por capas convierte la operación en una secuencia razonada en lugar de una búsqueda desordenada.

C.11 Criterios de aceptación final

El proyecto puede considerarse cerrado cuando cumple criterios funcionales, técnicos y documentales. En la parte funcional, la API debe permitir bootstrap, login, gestión básica de usuarios, transportes, vehículos, conductores, asignaciones, estados y eventos. Además, los endpoints protegidos deben respetar autenticación y roles, y los errores deben mantener códigos HTTP coherentes.

En la parte técnica, el repositorio debe compilar, los tests deben pasar, la imagen Docker debe construirse y el contenedor debe ejecutarse con usuario no root. Terraform debe estar formateado y validar correctamente. La infraestructura debe poder aplicarse en dev, ejecutar migraciones, escalar ECS, responder por HTTPS y registrar logs correlables. Si se realiza una validación AWS final, debe terminar con destroy y auditoría de ausencia de recursos efímeros.

En la parte operativa, deben existir runbooks de despliegue, rollback, restore, logs, alarmas y destroy. El rollback debe poder explicarse como retorno a un tag de imagen conocido. El restore debe estar validado mediante snapshot manual y tarea de migración. El coste debe estar identificado y separado entre recursos efímeros y persistentes.

En la parte documental, la memoria debe describir el alcance real, no una intención futura. Los requisitos deben estar alineados con lo implementado, la arquitectura debe reflejar la cuenta y región usadas, y las evidencias deben estar listadas e incorporadas en las secciones correspondientes. Las limitaciones también forman parte del cierre: no hay frontend, no hay entorno productivo, no hay autoscaling, no hay WAF y no hay trazas distribuidas. Es preferible declarar estos límites con claridad que sugerir una madurez que no se ha implementado.

C.12 Uso de los TFG de referencia

Los TFG de referencia revisados muestran una estructura habitual en memorias técnicas: resumen, contexto, requisitos, diseño, implementación, validación, planificación, costes, conclusiones y anexos. También muestran que una memoria de entre 85 y 110 páginas no se consigue solo con describir código, sino desarrollando el razonamiento de diseño, las alternativas, la validación y el material complementario.

La ampliación de esta memoria sigue esa línea, pero adaptada a TransitOps. El contenido añadido se centra en explicar por qué se eligieron ASP.NET Core, PostgreSQL, Docker, Terraform, ECS Fargate, RDS, ALB, CloudWatch y GitHub Actions; cómo se diseñó el contrato

HTTP; cómo se gestiona configuración y secretos; cómo se opera un despliegue; y cómo se relacionan requisitos con evidencias.

No se incorporan textos de los TFG de referencia. Su utilidad es estructural: ayudan a calibrar la densidad esperada de una memoria final y a detectar que la versión anterior de TransitOps era demasiado compacta para el nivel de detalle que suele verse en trabajos entregables. La nueva versión mantiene el mismo proyecto, pero explica con mayor profundidad las decisiones tomadas.

C.13 Mantenimiento posterior

Aunque el TFG se cierra con un entorno dev efímero, el repositorio queda preparado para mantenimiento posterior. La primera tarea razonable sería repetir una validación AWS final si se necesitasen evidencias nuevas en una fecha posterior, siguiendo el mismo flujo de recreación, validación y destroy ya documentado.

La segunda tarea sería ejecutar una validación AWS final si se necesitan evidencias recientes. Esa validación debe seguir el flujo ya documentado: recrear dev, publicar imagen, ejecutar migración, escalar ECS, validar HTTPS y login, revisar observabilidad y destruir. No conviene dejar el entorno vivo solo por comodidad, porque el diseño del proyecto asume coste temporal controlado.

La tercera tarea sería revisar permisos IAM si el proyecto evolucionase más allá del TFG. El rol OIDC de GitHub es aceptable para un entorno académico y efímero, pero un entorno productivo debería restringir acciones, recursos y condiciones con mayor precisión. Del mismo modo, una evolución a producción exigiría backups RDS automáticos, retención de logs más definida, alarmas con notificación confirmada, WAF, rotación de secretos y posiblemente autoscaling.

La cuarta tarea sería ampliar la observabilidad. El proyecto ya permite correlacionar peticiones, pero no incluye trazas distribuidas ni métricas de negocio. Si TransitOps creciese, sería útil medir número de transportes creados, errores por endpoint, duración de operaciones críticas y eventos de dominio relevantes. Estas métricas permitirían pasar de una observabilidad centrada en infraestructura a una observabilidad también orientada al producto.

Finalmente, cualquier ampliación funcional debería respetar la orientación original del proyecto: mantener el dominio comprensible y no sacrificar calidad operativa por añadir casos de uso. La base actual permite crecer hacia frontend, producción o más automatización, pero el valor del TFG está precisamente en haber demostrado un camino completo y reproducible antes de ampliar alcance.

Comparativa estructural con memorias de referencia

D.1 Criterios observados

Para ajustar la memoria a una longitud y nivel de detalle más próximos a otros TFG técnicos, se revisaron varias memorias de referencia proporcionadas. La revisión se centró en estructura, densidad documental y tipos de contenido incluidos, no en copiar texto ni decisiones concretas. Los trabajos revisados presentan extensiones aproximadas entre 85 y 108 páginas, con una combinación de capítulos principales y anexos.

El patrón común observado es que la memoria no se limita a describir el resultado final. Normalmente dedica espacio a motivación, contexto tecnológico, requisitos, diseño, implementación, pruebas, planificación, costes, conclusiones y anexos. En proyectos con componente software, el diseño e implementación suelen ocupar una parte sustancial. En proyectos con despliegue o hardware, también se dedica espacio a entorno, validación y procedimientos.

La versión inicial de TransitOps concentraba mucha información en pocos capítulos. Técnicamente era coherente, pero explicaba de forma demasiado sintética decisiones que en una defensa conviene desarrollar: por qué se eligieron ciertas tecnologías, qué alternativas se descartaron, cómo se valida cada capa y cómo se opera el sistema una vez desplegado.

D.2 Ajustes aplicados a TransitOps

A partir de esa revisión, la memoria se amplió en tres direcciones. La primera fue añadir un marco tecnológico. Este capítulo permite explicar ASP.NET Core, PostgreSQL, Docker, Terraform, AWS, GitHub Actions, CloudWatch y Secrets Manager como decisiones conectadas con los objetivos del proyecto. Así, la memoria no presupone que el lector conozca por qué cada herramienta aparece en la arquitectura.

La segunda dirección fue separar arquitectura general y diseño detallado. La arquitectura describe la vista global del sistema, mientras que el diseño detallado explica contrato HTTP, dominio, persistencia, autenticación, configuración, despliegue, validación y decisiones descartadas. Esta separación hace que el documento sea más fácil de defender: primero se presenta la forma del sistema y después se justifican las decisiones internas.

La tercera dirección fue reforzar anexos y operación. Los anexos recogen trazabilidad, evidencias, procedimientos, checklist y criterios de aceptación. Este material es especialmente adecuado para TransitOps porque el proyecto tiene una orientación DevOps fuerte. El valor no está solo en la API, sino en demostrar que puede desplegarse, observarse, recuperarse y destruirse de forma controlada.

D.3 Resultado de la comparación

Tras la ampliación, la memoria queda más cercana a las referencias en extensión y estructura. El documento mantiene el mismo alcance funcional, pero desarrolla mejor el razonamiento técnico. Esto evita una ampliación artificial basada en repetir información y, en cambio, convierte decisiones ya tomadas durante los sprints en contenido explicativo.

La comparación también confirma la utilidad de apoyar la memoria en figuras, capturas y anexos. TransitOps incorpora evidencias visuales de las validaciones cloud y mantiene el detalle operativo en anexos para que el documento pueda leerse y defenderse sin depender de explicaciones externas.

El resultado final es una memoria más completa para entrega: conserva una narrativa clara, explicita decisiones, documenta limitaciones y proporciona material operativo suficiente para una defensa técnica. La longitud aumenta sin cambiar el proyecto ni inventar funcionalidades no implementadas.

Lista de acrónimos

ALB Application Load Balancer. [9](#), [19](#), [20](#)

API Application Programming Interface. [2](#)

APM Application Performance Monitoring. [10](#)

AWS Amazon Web Services. [1](#)

CI/CD Continuous Integration / Continuous Deployment. [2](#)

DNS Domain Name System. [19](#), [21](#)

ECR Elastic Container Registry. [20](#)

ECS Elastic Container Service. [20](#)

HTTPS HyperText Transfer Protocol Secure. [19](#), [21](#)

IAM Identity and Access Management. [9](#)

JWT JSON Web Token. [16](#), [18](#), [30](#), [44](#), [53](#)

OIDC OpenID Connect. [9](#)

RDS Relational Database Service. [20](#)

SSM Systems Manager. [9](#), [20](#)

TLS Transport Layer Security. [9](#), [19](#)

VPC Virtual Private Cloud. [20](#)

Glosario

backend Componente servidor responsable de exponer la lógica de negocio, gestionar la persistencia y ofrecer una interfaz de comunicación a otros clientes o sistemas.. [2](#)

infraestructura como código Práctica consistente en definir recursos de infraestructura mediante ficheros versionables y automatizables.. [1](#), [3](#)

stateless Propiedad de un servicio que no conserva estado de sesión en memoria entre peticiones y puede escalarse más fácilmente detrás de un balanceador.. [6](#), [19](#)

Bibliografía

- [1] HashiCorp, “Terraform documentation,” 2026, consultado el 25 de mayo de 2026. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>
- [2] The PostgreSQL Global Development Group, “Postgresql documentation,” 2026, consultado el 25 de mayo de 2026. [Online]. Available: <https://www.postgresql.org/docs/>
- [3] Docker, Inc., “Docker documentation,” 2026, consultado el 25 de mayo de 2026. [Online]. Available: <https://docs.docker.com/>
- [4] Amazon Web Services, “Amazon elastic container service documentation,” 2026, consultado el 25 de mayo de 2026. [Online]. Available: <https://docs.aws.amazon.com/ecs/>
- [5] —, “Amazon rds documentation,” 2026, consultado el 25 de mayo de 2026. [Online]. Available: <https://docs.aws.amazon.com/rds/>
- [6] —, “Elastic load balancing documentation,” 2026, consultado el 25 de mayo de 2026. [Online]. Available: <https://docs.aws.amazon.com/elasticloadbalancing/>
- [7] —, “Amazon route 53 documentation,” 2026, consultado el 25 de mayo de 2026. [Online]. Available: <https://docs.aws.amazon.com/route53/>
- [8] —, “Aws certificate manager documentation,” 2026, consultado el 25 de mayo de 2026. [Online]. Available: <https://docs.aws.amazon.com/acm/>
- [9] —, “Amazon cloudwatch documentation,” 2026, consultado el 25 de mayo de 2026. [Online]. Available: <https://docs.aws.amazon.com/cloudwatch/>
- [10] —, “Aws secrets manager documentation,” 2026, consultado el 25 de mayo de 2026. [Online]. Available: <https://docs.aws.amazon.com/secretsmanager/>
- [11] —, “Aws systems manager documentation,” 2026, consultado el 25 de mayo de 2026. [Online]. Available: <https://docs.aws.amazon.com/systems-manager/>

- [12] GitHub, "Github actions documentation," 2026, consultado el 25 de mayo de 2026.
[Online]. Available: <https://docs.github.com/actions>
- [13] Microsoft, "Asp.net core documentation," 2026, consultado el 25 de mayo de 2026.
[Online]. Available: <https://learn.microsoft.com/aspnet/core/>